XBRL GL Next Taxonomy Framework — Part 1: General rules

Document status: Draft — For Collaborative Review

Date: 2026-09-15

Author: Nobuyuki SAMBUICHI

Role: Advisor to XBRL Japan

Project: XBRL GL Next

Distribution: Public collaborative review draft.

Repository version: public collaborative review draft.

Editors: SAMBUICHI, Nobuyuki

Contributors: invited project participants (individual names not yet confirmed for publication).

This document is a draft under development by Nobuyuki SAMBUICHI, Advisor to XBRL Japan, for collaborative review within the XBRL GL Next project. It is made available for public collaborative review. Copyright and license terms under review.

Status

This document is a draft under development by Nobuyuki SAMBUICHI, Advisor to XBRL Japan, for collaborative review within the XBRL GL Next project. It is made available through a public GitHub repository for collaborative review. It has not been approved or endorsed as an official specification by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Abstract

This document specifies the serialisation-independent semantic architecture and general modelling rules for the XBRL GL Next Taxonomy Framework. It preserves the proven semantic, modular, audit-trail, and traceability capabilities of the 2015 framework while enabling more flexible domain and module extension with stronger comparability.

- It defines the **FSM**, **BSM**, and **LHM** as a common, serialisation-neutral semantic and logical model; **Specialisation** as the deterministic **FSM**-to-**BSM** process; and **Graph Walk** as the deterministic **BSM**-to-**LHM** process;
- It defines modular assembly, Shared, Aligned, and Distinct extension, taxonomy assembly, and post_graphwalk processing without limiting future implementations to the predefined 2015 case combinations; and
- It provides the common semantic basis from which both the XBRL 2.1 tuple-based XML realisation and OIM-compatible realisations, including xBRL-CSV, can preserve equivalent business meaning, identity, logical relationships, and traceability.

Binding-specific constructs are outside the semantic-model contracts defined in this Part. Parts 2 and 3 specify the XBRL 2.1/XML and OIM-compatible realisations respectively.

Table of Contents

Status 1

Abstract 1

1 Language Independence 5

2 Introduction 5
- **2.1 Scope of this document** 5
- **2.2 Normative language and document conventions** 5
- **2.3 Relationship to other Parts** 5

3 Terms and definitions 6
- **3.1 General concepts** 6
- **3.2 Semantic-model constructs** 6
- **3.3 Semantic-model layers and lifecycle artefacts** 7
- **3.4 Semantic processing** 8
- **3.5 Review, conformance, and taxonomy completion** 9
- **3.6 Modules and taxonomy assembly** 10

4 Requirements and Motivation 11

5 Framework Architecture Overview 13
- **5.1 Semantic-model layers and processing pipeline** 13
- **5.2 Processing pipeline** 15
  - **5.2.1 Specialisation (FSM → BSM)** 15
  - **5.2.2 Graph Walk (BSM → LHM)** 15
- **5.3 FSM, BSM, and LHM definition tables** 16
  - **5.3.1 General** 16
  - **5.3.2 Foundational Semantic Model (FSM)** 16
  - **5.3.3 Business Semantic Model (BSM)** 17
  - **5.3.4 Multiple inheritance in the BSM** 18
  - **5.3.5 Logical Hierarchical Model (LHM)** 18
  - **5.3.6 Relationship among the three tables** 20
  - **5.3.7 LHM review after Graph Walk** 21
  - **5.3.8 Post-Graph Walk processing** 22
  - **5.3.9 HMD for taxonomy definition table** 23
  - **5.3.10 Relationship among candidate LHM, LHM reviewed, and HMD for taxonomy** 25
  - **5.3.11 Datatype binding and Value Domain generation** 26
  - **5.3.12 Class hierarchy and occurrence identity** 26
- **5.4 Modules and taxonomy assemblies** 27
  - **5.4.1 Module types** 27
  - **5.4.2 Use-case taxonomy assemblies** 27
  - **5.4.3 Semantic and syntax bindings** 27
  - **5.4.4 `gen` module and Value Domains** 27
- **5.5 XBRL QName and concept naming** 28
  - **5.5.1 XBRL concepts and QName** 28
  - **5.5.2 Prohibited and corrected example** 29
  - **5.5.3 `local_name` reuse and duplicate checks** 29
- **5.6 Semantic Path** 29
  - **5.6.1 Purpose and Reference** 29
  - **5.6.2 Syntax Rules and Construction** 29
  - **5.6.3 Functional Role in the Framework** 30
  - **5.6.4 Conformance and Validation** 30

6 Extension Mechanisms and Governance 31
- **6.1 Semantic authority and extension point** 31
- **6.2 Shared, Aligned, and Distinct** 31

6.3 Namespace, ownership, and versioning **31**
6.4 Property matching, duplicates, and conflicts **31**
6.5 Aligned Pool governance **31**
7 Specialisation and Graph Walk **32**
7.1 Scope, terminology, and processing stages **32**
7.2 Specialisation: FSM to BSM **35**
7.2.1 Inputs and registration **35**
7.2.2 Semantic identifier and Class-reference resolution **35**
7.2.3 Inheritance resolution **35**
7.2.4 Property matching and duplicate-check columns **38**
7.2.5 Override, addition, and deletion **38**
7.2.6 Abstract Classes and invalid Associations **38**
7.2.7 BSM output contract **38**
7.2.8 Specialisation conformance **39**
7.3 Graph Walk: BSM to LHM **39**
7.3.1 Inputs and root selection **39**
7.3.2 Class map and property order **39**
7.3.3 Depth-first traversal **39**
7.3.4 Association semantics **41**
7.3.5 Cycles and repeated occurrences **41**
7.3.6 Semantic path **41**
7.3.7 Boundary between Graph Walk and post_graphwalk processing **41**
7.3.8 LHM output contract **42**
7.3.9 Graph Walk conformance **42**
7.4 Later root-specific syntax binding (HMD) **42**
7.5 Implementation independence **42**
7.6 Conformance test coverage (informative) **43**
8 Modules and the Palette Model **43**
8.1 Module concept **43**
8.2 Baseline module composition **43**
8.3 Module dependencies and ownership **43**
8.4 Taxonomy assembly realisation **43**
8.5 Palette model **44**
9 Palette Taxonomy **44**
9.1 Purpose **44**
9.2 Tuple palette taxonomy **44**
9.3 OIM-oriented taxonomy **44**
9.4 Message and DTS entry points **44**
10 How to create extensions within the XBRL GL Next Taxonomy Framework **45**
10.1 Define the semantic extension **45**
10.2 Validate and specialise **45**
10.3 Generate hierarchies **45**
10.4 Generate taxonomy bindings **45**
11 Syntax Binding **45**
11.1 Binding source **45**
11.2 Binding information **45**
11.3 XBRL 2.1 and OIM equivalence **45**
11.4 Validation **46**
12 Reverse engineering and migration support (informative) **46**
12.1 Purpose **46**
12.2 Background: XBRL GL XML (XBRL 2.1 tuple-based realization) and implicit semantics **46**

12.3 Principle of semantic primacy **46**
12.4 Role of the Foundational Semantic Model (FSM) **46**
12.5 Reconstruction of semantic layers **47**
12.6 Regeneration and dual syntax support **47**
12.7 Summary **47**
13 Conformance **47**
13.1 General **47**
13.2 FSM conformance **47**
13.3 BSM conformance **48**
13.4 LHM conformance **48**
13.5 Execution-record and post_graphwalk conformance **48**
13.6 Processing conformance **48**
Appendix A (informative) Example: Extending "Sales Order" and "Purchase Order" from abstract class "Order" **49**
A.1 Two-phase architecture **49**
A.1.1 Specialisation Processing **49**
A.1.2 Graph Walk Processing **50**
A.2 Abstract class 'Order' and its specialisation **50**
Appendix B (informative) Example of the normal navigation Graph Walk **53**
B.1 Graph Walk: Defining the LHM by copying the BSM **53**
B.2 The Graph Walk process for the root class A **54**
Appendix C (informative) Semantic Layers Example: defining an extension in the FSM and generating the resulting BSM and LHM **57**
C.1 Purpose and scope **57**
C.2 Inputs (example sheets) **57**
C.3 Design policy used in the example **57**
C.4 Step-by-step procedure (FSM → BSM → LHM) **58**
C.4.1 Step 1 — Define the extension in the FSM **58**
C.4.2 Step 2 — Apply Specialisation to create the BSM **58**
C.4.3 Step 3 — Generate the LHM by deterministic Graph Walk **58**
C.5 Conformance checks recommended for this workflow **59**
C.6 Expected outputs and how they are used **59**
Appendix D (informative) Example: "VAT category taxable amount" (EN 16931 BT-116) **60**
D.1 UBL 2.1 (Peppol BIS Billing 3.0 – UBL Invoice) **60**
D.2 UN/CEFACT CII D16B (EN 16931 CII) **60**
D.3 Example syntax binding table entry **60**
D.4 Syntax binding example: XBRL GL tuple palette taxonomy **61**
Appendix E References **63**
Appendix F Intellectual property status **64**
Appendix G Document history **65**
Appendix H Errata corrections in this document **69**

1 Language Independence

This collaborative review draft is written in English, and the preferred spelling convention is UK English.

2 Introduction

2.1 Scope of this document

This document specifies the serialisation-independent technical and modelling rules needed to satisfy the XBRL GL Next Requirements Specification. It defines one common semantic and logical model that supports both the XBRL 2.1 tuple-based XML realisation and OIM-compatible realisations without making either physical architecture authoritative.

This document defines general rules for the organisation and modelling of XBRL GL taxonomy artefacts and their extension mechanisms. It specifies design rules for:

- layered semantic modelling and deterministic hierarchy generation;
- modular packaging, palette-based taxonomy assembly, and jurisdiction-specific extension; and
- Aligned cross-jurisdiction extension governance, including the aligned extension registry, aligned semantic library, and Aligned Pool.

This document does not define jurisdiction-specific filing rules, authority-specific submission interfaces, or operational exchange procedures. Such requirements are expected to be expressed through jurisdiction modules and the applicable binding-specific artefacts.

2.2 Normative language

The key words "MUST", "MUST NOT", "SHOULD", "SHOULD NOT", and "MAY", when they appear in uppercase, are interpreted as described in RFC 2119. "MUST" and "MUST NOT" express requirements indispensable to artefact conformance or interoperability. "SHOULD" and "SHOULD NOT" express recommended courses for which a conforming implementation can use another documented course when justified. "MAY" expresses a permitted option. Other requirement language is descriptive.

2.3 Relationship to other Parts

The XBRL GL Next Requirements Specification provides the background, design objectives, and requirements for the Framework.

The XBRL GL Next Taxonomy Framework consists of the following Parts:

- XBRL GL Next Taxonomy Framework - Part 1: General rules;
- XBRL GL Next Taxonomy Framework - Part 2: Tuple palette taxonomy;
- XBRL GL Next Taxonomy Framework - Part 3: OIM palette taxonomy;
- XBRL GL Next Taxonomy Framework - Part 4: Aligned Pool and extension governance;
- XBRL GL Next Taxonomy Framework - Part 5: Tavi palette taxonomy.

This Part 1 defines the serialisation-independent semantic pipeline and the formal HMD contract. Part 2 defines the XBRL 2.1 Tuple realisation generated from formal HMDs. Part 3 defines the OIM realisation generated from the same HMDs. Part 4 governs extension classification, registration, alignment and publication. Part 5 defines purpose-specific HMD views and their Tavi taxonomy and Table Template realisation.

This Part 1 establishes the common semantic modelling and processing rules for the Framework, from FSM through BSM and LHM to formal HMDs, providing the serialisation-independent foundation for the taxonomy bindings defined in the other Parts.

Parts 2, 3 and 5 provide alternative taxonomy realisations of the common semantic framework established by Part 1, under the extension governance defined in Part 4. Each HMD serves as a purpose-specific palette, selecting reusable semantics from COR, BTX and AAA, which constitute the semantic paint boxes.

3 Terms and definitions

For the purposes of this document, the following terms and definitions apply. These terms provide the common terminology used by this Part and, unless otherwise specified, by the other Parts of the XBRL GL Next Taxonomy Framework. Compatibility mappings may document legacy or implementation-specific labels.

3.1 General concepts

normative
describes a provision with which conformance is required. The key words defined in 2.2 indicate the degree of obligation.

informative
describes explanatory material, examples, implementation notes, or other material that does not establish a conformance requirement.

QName
a namespace-qualified name consisting of a namespace URI and a local name. A prefix is a document-level alias for the namespace URI. QName is used for syntax binding and taxonomy concepts; semantic-model identity is expressed by a semantic-model identifier.

3.2 Semantic-model constructs

Class
a semantic-model construct that defines a type of business object and owns or inherits properties.

property
an Attribute or Association declared for, or inherited by, a Class.

Attribute
a property that represents a value and its datatype rather than a relationship to another Class.

Association
a property that relates a source Class to a target Class.

association type
the semantic classification of an Association as Specialisation, Composition, Aggregation, or Reference. In the **FSM** and **BSM** file contracts, it is represented by `property_type`.

semantic-model identifier
a syntax-independent identifier that establishes the identity of a Class, Attribute, or Association. Its components depend on the item type.

Class semantic-model identifier
the combination of the module identifier and Class term that identifies a Class.

Attribute semantic-model identifier
the combination of the Class semantic-model identifier and property term that identifies an Attribute of that Class.

Association semantic-model identifier
the combination of the source Class semantic-model identifier, association role, and target Class semantic-model identifier that identifies an Association.

association role
the term that identifies the role of a target Class in relation to its source Class and distinguishes multiple Associations between the same Classes.

associated Class
the target Class of an Association. In an **FSM** or **BSM** file, its Class semantic-model identifier is represented by `associated_module` and `associated_class`.

multiplicity
the minimum and maximum permitted number of occurrences of a property. In an **FSM** Specialisation declaration, `0` or `0..0` may express a deletion directive. In an **LHM reviewed**, `0..0` expresses a permitted non-use restriction; the row is retained for traceability.

3.3 Semantic-model layers and lifecycle artefacts

Foundational Semantic Model
FSM
the serialisation-neutral semantic-model layer in which applicable Shared, Aligned, and Distinct definitions, modules, Classes, properties, Specialisation Associations, candidate root Classes, constraints, and provenance information are declared before inheritance is resolved.

FSM source
the authoritative authored input artefact comprising one or more conformant **FSM** files supplied to Specialisation.

Business Semantic Model
BSM
the effective serialisation-neutral semantic-model layer in which inheritance, additions, overrides, multiplicity constraints, and deletion directives have been resolved. Composition, Aggregation, and Reference Associations remain unresolved and available for **Graph Walk**.

BSM result
the generated **BSM** artefact produced directly by Specialisation from the applicable **FSM source**. It is not modified manually.

Logical Hierarchical Model
LHM
the logical model layer that represents the deterministic hierarchical occurrences produced from a conformant **BSM** by **Graph Walk**. It records row order, hierarchy, source traceability, **semantic paths**, and initial taxonomy-preparation values in the 18-column LHM contract.

candidate LHM
the generated LHM artefact produced directly by **Graph Walk** before semantic-model review and approval. It includes initial `local_name` and `xpath` values and is not modified manually.

LHM reviewed
a separately identifiable controlled review artefact prepared from a semantically approved **candidate LHM**. The responsible model definer assigns reviewed `value_domain` identifiers to applicable Attribute occurrences, assigns or adjusts binding-specific `local_name` values where required by occurrence position, and may apply only the permitted business-requirement-based restrictions to `multiplicity`. Every such review decision is recorded as required by this document.

HMD for taxonomy
the set of formal 18-column **HMD for taxonomy** files produced from an approved **LHM reviewed** after all **Post-Graph Walk** checks have passed, with one formal **HMD** for each effective `level = 1` root. Each formal **HMD** preserves the reviewed `local_name` and `multiplicity` values of every effective row, regenerates `xpath`, and preserves all other LHM values. An occurrence whose reviewed multiplicity is `0..0`, together with its occurrence subtree, is excluded.

Hierarchical Message Definition
HMD
a root-specific syntax-binding artefact derived from an **HMD for taxonomy** when required by the selected syntax binding.

HMD root
the root Class selected from an **HMD for taxonomy** to serve as the root of a root-specific syntax binding. In the semantic model it is identified by its Class semantic-model identifier; the selected syntax binding assigns the applicable QName.

3.4 Semantic processing

Specialisation Association
an Association whose association type is Specialisation and that identifies a superclass from which the specialising Class inherits properties. In an **FSM** file, this association type is represented by the value `Specialisation` in `property_type`.

Composition Association
an Association whose association type is Composition. **Graph Walk** emits the associated Class occurrence and its reachable effective properties as part of the containing hierarchy.

Aggregation Association
an Association whose association type is Aggregation. **Graph Walk** emits the associated Class occurrence and its reachable effective properties while preserving the Association semantics through `source_bsm_id`.

Reference Association
an Association whose association type is Reference. **Graph Walk** emits the reference occurrence and the target Class Attributes marked `identifier=PK` as `REF`. A missing primary key is reported as a non-fatal validation result.

Specialisation
the deterministic **FSM**-to-**BSM** process that resolves only Specialisation Associations, including the resulting inheritance, additions, overrides, multiplicity constraints, and deletion directives. All other Association types remain unresolved in the **BSM** for subsequent **Graph Walk** processing.

active inheritance path
the ordered sequence of Classes currently being resolved during Specialisation, used to detect and report a Specialisation cycle. Its internal representation is implementation-specific.

Graph Walk
the deterministic depth-first **BSM**-to-**LHM** process that starts from one or more supplied root Classes and uses last-in, first-out processing. Composition and Aggregation Associations expand the candidate hierarchy. Reference Associations emit the prescribed reference rows without recursively processing the referenced Class.

LIFO list
the abstract last-in, first-out processing list used to define **Graph Walk** order. An implementation may realise the list by recursion or another mechanism only when the observable PUSH, POP, traversal, and output order is equivalent.

active traversal path
the ordered sequence of Classes currently being traversed from a root. It is used to identify a traversal cycle and is independent of an occurrence name.

PUSH
the action of adding a Class traversal state to the top of the **LIFO list**.

POP
the action of removing the completed Class traversal state from the top of the **LIFO list**.

occurrence
a context-specific appearance of a **BSM** Class or property in an LHM or subsequent binding artefact. Each LHM occurrence is identified by its `semantic_path`. The same `source_bsm_id` may appear in multiple occurrences having different **semantic paths**, and those occurrences may be assigned different binding-specific `local_name` values and QNames.

occurrence-key Class
a Class whose instances require explicit occurrence identity in an instance-oriented representation. The root Class and every Class whose effective maximum occurrence is greater than one are **occurrence-key Classes**. A non-root Class with effective multiplicity `0..1` or `1..1` is not an **occurrence-key Class**.

Structured CSV
occurrence-key column a column that identifies occurrences of one **occurrence-key Class**. Such a column is derived from the authoritative HMD root status and multiplicity; it is not created for every Class and is not a **Value Domain** representation.

semantic path
a stable, human-readable LHM row identifier derived from the ordered hierarchy of occurrence `name` values. It identifies semantic context independently of a serialisation. A later adjustment to `local_name` does not alter `name` or `semantic_path`.

post_graphwalk
the deterministic **Post-Graph Walk** stage that processes an approved **LHM reviewed**, regenerates `xpath` from each row's `module` prefix and current `local_name`, performs the **Post-Graph Walk** checks, and produces the **HMD for taxonomy** while preserving the reviewed `local_name`, `multiplicity`, and input artefact.

3.5 Review, conformance, and taxonomy completion

responsible model definer
the person or body accountable for reviewing a **candidate LHM**, determining whether permitted adjustments and restrictions are justified, recording those changes, and approving the resulting LHM artefacts.

semantic-model error
an error in semantic content, structure, hierarchy, occurrence relationships, source traceability, or another semantic-model requirement that requires correction of the applicable **FSM source** and rerunning **Specialisation** and **Graph Walk**. A `local_name` collision and a permitted business-requirement-based multiplicity restriction are not, by themselves, semantic-model errors.

post-Graph Walk checks
the technical checks performed by post_graphwalk on an **LHM reviewed**, including `local_name` NCName validity, (`module`, `local_name`) uniqueness within each root-specific **HMD**, XPath regeneration and uniqueness within each **HMD**, consistency of any QName deliberately reused across **HMDs**, permitted multiplicity restrictions, and preservation of all values other than regenerated `xpath`.

generated taxonomy
the direct output of a **conformant taxonomy generator** before confirmation or completion of all applicable binding-specific schema requirements.

conformant taxonomy generator
the software implementation that generates an XBRL GL Next taxonomy from one or more formal **HMDs** in accordance with the applicable taxonomy-generation requirements.

binding-specific schema completion
the controlled manual process, where required, of adding declarations or restrictions that are specific to the selected syntax binding and are not represented in the **HMD for taxonomy**.

completed taxonomy
the separately identifiable final release artefact that satisfies all applicable binding-specific schema requirements, whether or not manual binding-specific schema completion was required.

3.6 Modules and taxonomy assembly

module
a versioned, namespace-owned unit of reusable semantic definitions and the corresponding taxonomy components. A module boundary expresses ownership and dependency and is not merely a directory boundary.

Shared
the reuse category for concepts and structures whose meaning is valid across countries, regimes, industries, and implementations.

Aligned
the reuse category that specialises or constrains Shared definitions to maintain alignment with a stated international, regional, national, legal, regulatory, or public industry context.

Aligned Pool
a governed, versioned, and machine-readable collection of reusable Aligned semantic definitions. It comprises an aligned extension registry and an aligned semantic library.

aligned extension registry
the machine-readable governance record for the identity, provenance, alignment source, maintainer, status, version history, and artefact references of definitions submitted to or published in the Aligned Pool.

aligned semantic library
the machine-readable collection containing the accepted Aligned **FSM** definitions referenced by the aligned extension registry.

Distinct
the extension category for definitions specific to an enterprise, enterprise group, product, bilateral relationship, or closed community.

Palette model
the taxonomy implementation approach in which reusable module taxonomies are assembled into one or more message entry points without duplicating their authoritative semantic definitions.

message entry point
the physical XBRL taxonomy entry point that realises one root-specific hierarchy and loads the taxonomy components required for that hierarchy.

syntax binding
the rules and artefacts that map a semantic model or hierarchical model to an XBRL 2.1, xBRL-CSV, xBRL-JSON, or other physical serialisation.

taxonomy entry point
an XBRL schema or other DTS entry artefact through which a taxonomy assembly is discovered. In this framework, a message entry point is the root-specific taxonomy entry point corresponding to one **HMD**.

Value Domain
a governed permitted semantic value set for an Attribute, identified independently of the Attribute's semantic datatype and of any physical code-list distribution.

Value Domain identifier
a stable identifier that distinguishes one governed **Value Domain** from every other **Value Domain** across versions, authorities, and categories.

legacy Value Domain
a separately identified controlled vocabulary retained for compatibility with an earlier XBRL GL definition, including XBRL GL 2003-compatible code and value domains.

standard Value Domain
a separately identified **Value Domain** governed by an external authority or standard and qualified, where applicable, by its code-list identifier, edition or version, subset, provenance, and rights status.

user-defined Value Domain
a separately governed **Value Domain** defined for an enterprise, community, product, or other user-controlled scope without assuming the identity of a standard **Value Domain**.

Value Domain crosswalk
a governed mapping between members of two separately identified **Value Domains** that classifies each correspondence without replacing either **Value Domain**.

`gen` module
the shared, binding-neutral semantic module, and its corresponding generated taxonomy module, for reusable basic semantic types, governed **Value Domains**, and other reusable definitions required by multiple modules and by both Tuple and OIM bindings.

4 Requirements and Motivation

The XBRL GL taxonomy framework is designed to satisfy functional requirements including:

a) business documents (order/purchase, shipment/logistics, invoicing, payments) with at least a three-layer structure: document header, tax category totals, line items;
b) accounting books (daily journal/journal entries, trial balance, general ledger, AP/AR ledgers) including adjustments, cash application, and cash payment transactions;
c) monetary amounts and (sub)totals at appropriate levels, including line totals, taxable/basis amounts, tax amounts by category and/or type, paid amounts, and payable/outstanding amounts;
d) statistical and other non-financial records captured at transaction, document, and/or measurable-item level, including (as applicable) work time and attendance, inventory movement and quantities, production or service metrics, payment statistics, logistics/performance measures, and environmental/ESG measures; and
e) cross-cutting capabilities: multi-currency, business segment and accounting segment classification, system user activity logging (audit trail), and drill-down/roll-up to/from annual report structures (e.g., BS/P&L).

The framework defines a common serialisation-neutral semantic and logical model that can be realised consistently through XBRL 2.1 tuple-based XML and OIM-compatible serialisations. The model uses the **FSM**, **BSM**, and **LHM** layers and supports Shared, Aligned, and Distinct definitions.

The framework combines flexible extension with comparability. Reusable cross-jurisdiction definitions are governed through the Aligned Pool, while the Palette model supports deterministic modular taxonomy assembly without changing the authoritative semantic model.

These proposed requirements are defined in the companion “XBRL GL Next Requirements Specification” and are referenced here only to motivate the framework architecture and module boundaries.

Table 1 maps the requirements from the Requirements Specification to their support and implementation in this part of the framework.

Table 1 — Mapping of XBRL GL Next Requirements

Requirement ID	Requirement Name	Supported in Part 1 (General Rules)
Part 1-specific detail:	Common logical model for XBRL 2.1 and OIM	Defines one serialisation-neutral semantic and logical model preceding both bindings. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	Authoritative identity and traceability	Defines BSM `id`, LHM `source_bsm_id`, Semantic Path, and candidate-LHM identity checks. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	Specialisation processing	Resolves only Specialisation Associations and applies inheritance, additions, overrides, multiplicity constraints, and deletions. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	Graph Walk processing	Defines deterministic depth-first LIFO processing and Association-specific traversal rules. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	Candidate LHM review and approval	Distinguishes the Graph Walk output from a candidate LHM that has passed semantic-model review and conformance checks. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	Separation of post-Graph-Walk processing	Keeps permitted review adjustments and binding additions outside the candidate LHM. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
	No tuple dependency in the common model	Treats tuple nesting as a Part 2 realisation rather than the source of hierarchy. (No independent GL-REQ identifier is assigned in the Requirements Specification.)
GL-REQ-0001	Semantic layer separation	Defines **FSM** and applicable extensions as authoritative inputs and **BSM**/**LHM** as deterministic generated results.
GL-REQ-0009	Semantic stability of core concepts	Keeps Shared **FSM** definitions unchanged and resolves child-Class extension through Specialisation.
GL-REQ-0011	Interoperable extension mechanism for domain attributes	Defines explicit Shared, Aligned, and Distinct extension with identity, provenance, and traceability.
GL-REQ-0109	Business transaction module root and anchoring	Allows supplied domain roots without requiring the Accounting Entries root.
GL-REQ-0201	Modular taxonomy architecture	Supports deterministic modular taxonomy assembly without limiting combinations to the 2015 cases.
GL-REQ-0202	Cross-jurisdiction alignment	Provides semantic identity, Aligned reuse, and conflict checks for comparison.
GL-REQ-0204	Localisation ownership, governance, and aligned extension registry	Defines the Aligned Pool, aligned extension registry, aligned semantic library, and Part 4 boundary.
GL-REQ-0210	Aligned pool of reusable semantic components	Defines reusable Aligned Pool semantic components and its governance boundary in Clauses 6.5 and 8.

Requirement ID	Requirement Name	Supported in Part 1 (General Rules)
GL-REQ-0301	Support for XML and OIM	Provides the common model used by both XBRL 2.1/XML and OIM-compatible bindings.
GL-REQ-0302	OIM capability completeness	Defines serialisation-neutral input and separation from OIM binding; detailed OIM completeness is specified in Part 3.
GL-REQ-0303	Deterministic transformability	Requires bindings to preserve business meaning, logical relationships, identity, and traceability.
GL-REQ-0403	Interoperability evidence and pilot criteria	Defines deterministic semantic processing and traceability that provide inputs to interoperability evidence; pilot criteria are specified in the Requirements Specification.

5 Framework Architecture Overview

5.1 Semantic-model layers and processing pipeline

The semantic model consists of the three layers shown in **Figure 1**.

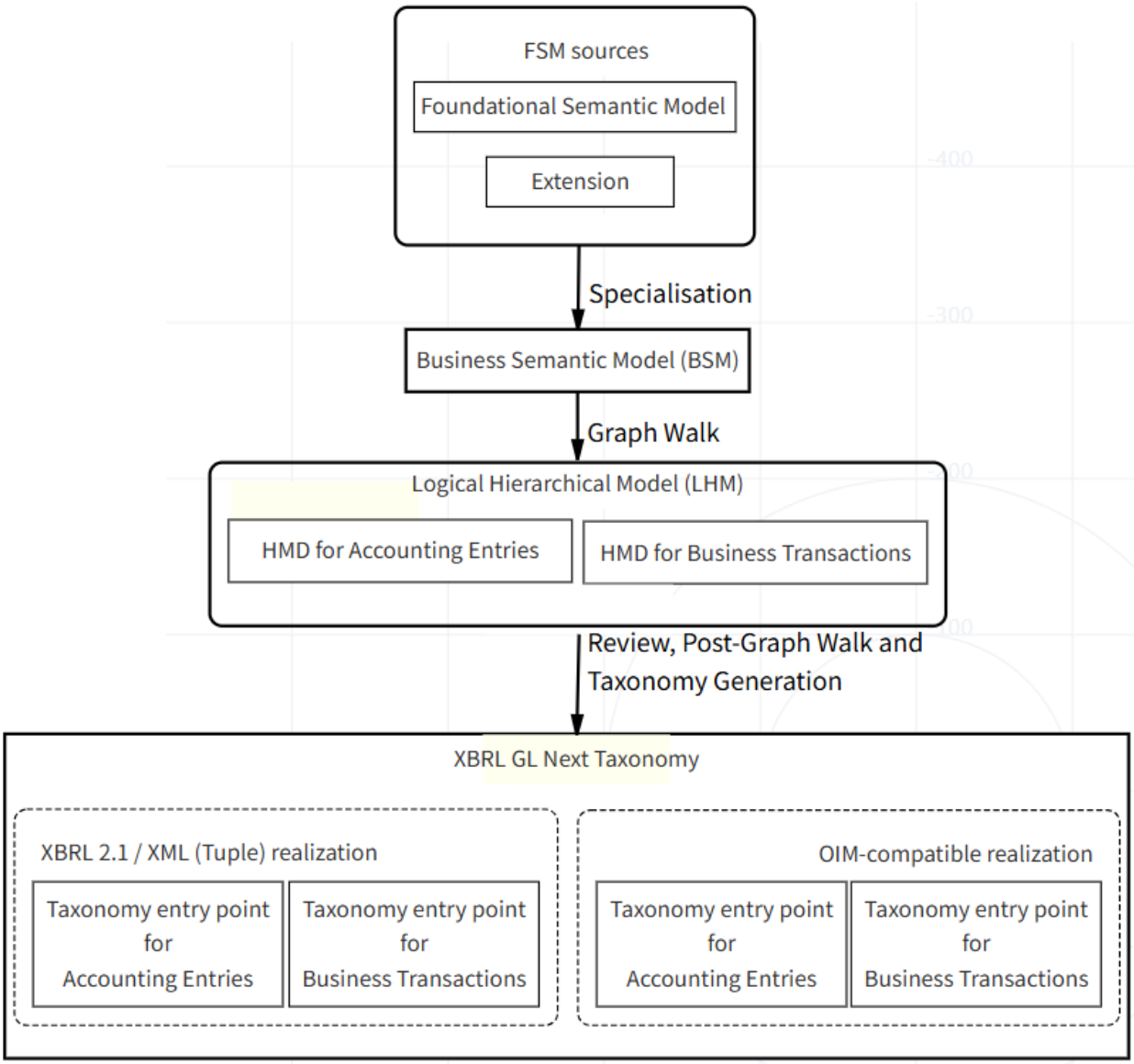


Figure 1 — Layered Semantic Model

a) Foundational Semantic Model (FSM)

The **FSM** is the authoritative semantic input. It may contain Shared, Aligned, and Distinct definitions and applicable extensions for one or more business domains.

Shared **FSM** definitions remain unchanged when used by an extension. An Aligned or Distinct child Class may specialise a Shared Class and declare additions, overrides, multiplicity constraints, and deletion directives. Specialisation resolves those declarations into the effective child definition in the **BSM**.

b) Business Semantic Model (BSM)

The **BSM** is the deterministic effective model produced by resolving only Specialisation Associations across the applicable **FSM** inputs. It contains the resulting inheritance, additions, overrides, multiplicity constraints, and deletion results and is not independently selected or modified after generation.

c) Logical Hierarchical Model (LHM)

Graph Walk generates a complete **candidate LHM** from the conformant **BSM** and supplied root Classes, including initial `local_name` and `xpath` values for every row. The candidate may be semantically approved only after the prescribed semantic-model and structural conformance checks

succeed. Subsequent permitted `local_name` adjustments, business-requirement-based `multiplicity` restrictions, XPath regeneration, and binding-specific processing occur in separate artefacts.

An LHM row is identified by `semantic_path`; `source_bsm_id` records the originating **BSM** `id` for traceability and validation. A semantically approved **candidate LHM** satisfies the 18-column contract and the correspondence conditions in 5.3.6.

The semantic-model layers are the **FSM**, **BSM**, and **LHM**. **HMD**, XSD, linkbases, OIM metadata, and instance layouts are syntax-binding artefacts.

Artefact	Role	Authority and derivation
FSM	Declares modules, Classes, properties, inheritance, Associations, roots, multiplicities, value domains, and extensions.	Authoritative semantic source edited by model authors.
BSM	Contains the effective concrete Classes and properties after inheritance, additions, overrides, multiplicity constraints, and deletion directives have been resolved. Non-Specialisation Associations remain unresolved.	Generated deterministically from the applicable conformant **FSM** inputs by Specialisation; not independently selected or modified.
LHM	Represents the candidate logical hierarchy produced by **Graph Walk** from a conformant **BSM** and supplied root Classes, including initial local_name and xpath values.	Generated deterministically from the **BSM result**; may be semantically approved only after the prescribed semantic-model and structural checks succeed.
HMD	Provides a root-specific hierarchy for a document, ledger, transaction, or report binding.	Derived from the **HMD for taxonomy** when required by the selected syntax binding; not a semantic-model layer.
Taxonomy	Implements an HMD and its reviewed semantic-model rows and meanings in XBRL 2.1 or OIM-oriented XBRL constructs.	Generated syntax-binding artefact.
Palette model	Provides reusable module taxonomies and fixed rules for assembling them into one or more DTS entry points.	XBRL implementation model; it does not author semantic content independently of the **FSM**.

The **FSM** is the semantic source for modules, Classes, properties, root candidates, and constraints. Root Classes are supplied to **Graph Walk**. Preparation of the **LHM reviewed** and taxonomy generation then follow the fixed rules of this Framework and the applicable binding Parts. No additional run-configuration artefact is required by this Part.

5.2 Processing pipeline

5.2.1 Specialisation (FSM → BSM)

Specialisation resolves only Specialisation Associations and applies inheritance, additions, overrides, multiplicity constraints, and deletion directives. The resulting **BSM** retains effective Composition, Aggregation, and Reference Associations unresolved for **Graph Walk**. The detailed requirements are specified in 7.2.

5.2.2 Graph Walk (BSM → LHM)

Graph Walk uses deterministic depth-first, last-in-first-out processing to transform a conformant **BSM** and supplied root Classes into a candidate 18-column **LHM**. Composition and Aggregation expand the hierarchy; Reference Associations are emitted without recursively processing the referenced Class. **Graph Walk** also generates initial `local_name` and `xpath` values mechanically for every emitted row.

5.3 FSM, BSM, and LHM definition tables

5.3.1 General

The semantic model comprises three, and only three, model layers:

a) the Foundational Semantic Model (**FSM**);
b) the Business Semantic Model (**BSM**); and
c) the Logical Hierarchical Model (**LHM**).

The principal normative transformation path between these model layers is:

FSM source → Specialisation → BSM result → Graph Walk → candidate LHM

For clarity, this subclause shows only the principal transformation path. The subsequent semantic-model review and approval, preparation of the **LHM reviewed**, **post-Graph Walk** processing, production of the **HMD for taxonomy**, and taxonomy generation and completion paths are specified in the following subclauses and shown in **Figure 2**.

The **candidate LHM**, **LHM reviewed**, and **HMD for taxonomy** represent successive lifecycle states or artefacts of the LHM. They do not constitute additional semantic-model layers.

5.3.2 Foundational Semantic Model (FSM)

The **FSM** is the authoritative, declarative semantic source. It defines modules, Classes, Attributes, Associations, inheritance relationships, multiplicities, representation terms, definitions, and local-language labels before inheritance is resolved.

The **FSM** can contain abstract Classes, Specialisation Associations, additions, overrides, and deletion directives. Its 15-column contract contains authored semantic information.

The **FSM** definition table has the following 15 columns, in the stated order.

No.	Column	Definition
1	sequence	The stable source order of the row within the **FSM** input. It determines Class declaration order, property order, Specialisation declaration order, and deterministic processing order.
2	level	The structural level within the tabular **FSM**. A Class row normally has level `1`; a property or Association row owned by that Class normally has level `2`. This is an authoring-table level and is not the final hierarchy level used in the LHM.
3	property_type	The kind of row. Permitted semantic kinds include `Class`, `Abstract Class`, `Attribute`, `Specialisation`, `Composition`, `Aggregation`, and `Reference`. `Specialisation` identifies a superclass relationship processed only by Specialisation. The other Association kinds remain available for **Graph Walk**.
4	identifier	An optional semantic identifier-role marker. For example, `PK` identifies an Attribute that constitutes a primary identifier of its Class. It is not the generated identifier of the row.
5	module	The module that owns the Class. For a property row, it identifies the module of the owning Class. Together with `class_term`, it identifies that Class within the **FSM**.
6	class_term	The semantic term of the owning Class. A Class is identified by the combination of `module` and `class_term`. On a property row, it identifies the Class to which the property directive belongs.
7	property_term	The semantic name of an Attribute. It is populated for an Attribute and empty for a Class or Association row. Within an owning Class, inherited Attributes are matched by `property_term`.

No.	Column	Definition
8	association_role	The role played by an associated Class in the context of the owning Class. It is populated for Specialisation, Composition, Aggregation, and Reference Associations and empty for an Attribute. The role can distinguish multiple semantic uses of the same associated Class.
9	representation_term	The semantic representation or datatype of an Attribute, such as Identifier, Code, Date, Text, Amount, or Quantity. It is populated for an Attribute and empty for an Association.
10	associated_module	The module containing the Class referenced by an Association. Together with `associated_class`, it identifies the Association target. It is empty for an Attribute.
11	associated_class	The Class referenced by an Association. For a Specialisation Association, this is the superclass. For Composition, Aggregation, and Reference Associations, it is the Class subsequently processed by **Graph Walk**.
12	multiplicity	The permitted cardinality of the Class, Attribute, or Association. Values include `1`, `0..1`, `1..*`, and `0..*`. On an inherited property directive, `0` or `0..0` means that the inherited property is to be deleted and is not an effective multiplicity in the resulting **BSM**.
13	definition	The authoritative definition of the Class, Attribute, or Association in the framework's base language.
14	label_local	The reviewed local-language label corresponding to the semantic term or Association role.
15	definition_local	The reviewed local-language version of `definition`.

5.3.3 Business Semantic Model (BSM)

The **BSM** is the effective semantic model produced from one or more conformant **FSM** files by Specialisation.

Specialisation recursively resolves each superclass, combines inherited properties, and applies local additions, overrides, multiplicity constraints, and deletion directives. The resulting **BSM** contains effective concrete Classes and Attributes, while Composition, Aggregation, and Reference Associations remain unresolved for **Graph Walk**.

The **BSM** has the same first 15 columns as the **FSM** and adds `id` as its sixteenth column.

No.	Column	Definition in the BSM
1	sequence	The deterministic output order assigned after Specialisation. It reflects the order of concrete Classes and their effective properties in the generated **BSM**.
2	level	The **BSM** tabular level. A concrete Class row has level `1`; each effective Attribute or Association owned by that Class has level `2`.
3	property_type	The effective row kind: `Class`, `Attribute`, `Composition`, `Aggregation`, or `Reference`.
4	identifier	The effective identifier-role marker inherited or locally defined for the row, such as `PK` for a Class identifier Attribute.
5	module	The module of the effective owning Class. An inherited property is placed in the context of the specialising Class and therefore records the child Class's owning module.
6	class_term	The effective owning Class. Together with `module`, it identifies the Class whose resolved property set contains the row.
7	property_term	The effective Attribute name after inheritance, addition, override, and deletion processing. It is empty for Association rows.

No.	Column	Definition in the BSM
8	association_role	The effective role of a Composition, Aggregation, or Reference Association. It is retained for occurrence naming and path construction during **Graph Walk**.
9	representation_term	The effective representation or datatype of an Attribute. It is empty for an Association row.
10	associated_module	The resolved module of the Class targeted by an effective Association.
11	associated_class	The resolved Class targeted by an effective Association. The combination of `associated_module` and `associated_class` resolves to exactly one **BSM** Class.
12	multiplicity	The effective cardinality after inheritance and local directives have been resolved. Deletion values are applied during Specialisation and are absent from the effective **BSM**.
13	definition	The effective base-language definition after inheritance and any local override.
14	label_local	The effective local-language label after inheritance and any local override.
15	definition_local	The effective local-language definition after inheritance and any local override.
16	id	The unique, stable identifier assigned to the effective **BSM** row and copied by **Graph Walk** to **LHM** `source_bsm_id`. It is syntax-independent. Module prefixes and other syntax-specific naming rules are defined in Parts 2 and 3 and applied by **Post-Graph Walk** and the **conformant taxonomy generator**.

5.3.4 Multiple inheritance in the BSM

For a Class with more than one superclass, Specialisation MUST combine the resolved properties of all superclasses before applying the child Class local property directives.

Inherited property correspondence is determined by the applicable semantic-model identifier.

- `property_term` for an Attribute; and
- the combination of `association_role`, `associated_module`, and `associated_class` for an Association.

Where corresponding inherited properties have the same identity:

- identical inherited properties that represent the same semantic-model item are merged;
- corresponding Attributes with the same `representation_term` are merged;
- corresponding Associations with an empty `representation_term` are merged;
- the merged multiplicity is the least restrictive range encompassing the inherited multiplicities; and
- the lower bound is the lowest inherited lower bound and the upper bound is the highest inherited upper bound, with `*` treated as unbounded.

Where corresponding inherited Attributes have different `representation_term` values:

- a local child-Class directive can resolve the difference through override or deletion;
- an unresolved difference is reported as a conflict; and
- the resolved **BSM** contains the resulting effective Attribute.

Multiple inheritance is conformant when all inherited-property conflicts are resolved deterministically.

5.3.5 Logical Hierarchical Model (LHM)

The LHM is the candidate logical hierarchy generated from a conformant **BSM** and one or more supplied root Classes. Its semantic content is serialisation-neutral; its initial `local_name` and `xpath` columns are mechanically **generated taxonomy**-preparation values and do not define semantic identity.

A root Class is identified by the combination of its `module` and `class_term`. **Graph Walk** traverses Composition and Aggregation Associations depth-first and processes Reference Associations according to their identifier-reference semantics.

The LHM represents occurrences of Classes and properties in their hierarchical semantic context using the 18 columns defined in 5.3.5.

The LHM definition table has the following 18 columns, in the stated order.

No.	Column	Definition in the LHM
1	sequence	The deterministic row number assigned in **Graph Walk** output order. It records the exact depth-first order of emitted occurrences.
2	module	The module of the semantic Class or Attribute represented by the occurrence. For an associated-Class occurrence, this is normally the target Class's module.
3	level	The depth of the occurrence in the generated hierarchy. The root Class has level `1`; its direct Attributes and associated-Class occurrences have the next applicable level.
4	type	The LHM occurrence type. `C` identifies a Class occurrence produced for a root, Composition, or Aggregation; `A` identifies an Attribute occurrence, including a copied reference identifier; and `R` identifies a Reference occurrence. Composition and Aggregation remain distinguishable through the **BSM** row identified by `source_bsm_id`.
5	identifier	The identifier-role marker applicable to the occurrence. A normal identifier Attribute may retain `PK`. An identifier copied beneath a Reference occurrence is marked `REF`.
6	name	The human-readable semantic occurrence name. For a root, it is derived from `class_term`; for an Attribute, from `property_term`; and for an associated-Class occurrence, from `association_role` and `associated_class` according to the prescribed naming rule. The ordered hierarchy of `name` values is used to construct `semantic_path`; a later `local_name` adjustment does not change it. An LHM `name` is not required to be globally unique. It identifies the occurrence in human-readable semantic terms; occurrence identity is provided by `semantic_path`, and taxonomy QName uniqueness is determined separately from (`module`, `local_name`) within each root-specific HMD.
7	datatype	The effective Attribute datatype derived from the **BSM** `representation_term`. It is empty for Class and Reference occurrence rows.
8	multiplicity	The effective cardinality of the occurrence. For an associated-Class or Reference occurrence, it is derived from the incoming Association. For an Attribute, it is derived from the effective **BSM** Attribute.
9	association_role	The semantic role of the incoming Association for an associated-Class or Reference occurrence. It is empty for root and Attribute occurrences. It is syntax-independent and preserves Association context used in Semantic Path construction.
10	definition	The base-language definition carried from the originating **BSM** row or, where prescribed, the associated Class definition.
11	label_local	The reviewed local-language label carried from the originating **BSM** row or associated Class.
12	definition_local	The reviewed local-language definition carried from the originating **BSM** row or associated Class.
13	source_bsm_id	The `id` of the **BSM** row that caused the occurrence to be emitted. For a root occurrence, it is the root Class row's `id`; for an Attribute, the Attribute row's `id`; for a Composition, Aggregation, or Reference occurrence, the Association row's `id`; and for a copied reference identifier, the target identifier Attribute's `id`. It provides traceability and is not an independently assigned **LHM** identifier.

No.	Column	Definition in the LHM
14	semantic_path	The stable, human-readable identifier of the LHM occurrence. It begins with `$`, includes the root as its first segment, and records the ordered occurrence path. Each segment uses the applicable LHM `module` value, one underscore, and a non-empty term normalised to ASCII letters only in accordance with 5.6.2. An Association role is concatenated directly with the associated Class term where applicable. It is independent of XML, JSON, CSV, or XBRL serialisation.
15	associated_module	The module of the Class context represented by the row. For an associated-Class or Reference occurrence, it is the target Class module. For an Attribute, it identifies the module of the Class context containing that Attribute occurrence.
16	class_term	The Class context of the occurrence. For a root or associated-Class occurrence, it is that Class's term. For an Attribute, it is the term of the Class whose effective property set supplied the Attribute.
17	local_name	The provisional XML Schema NCName generated mechanically by **Graph Walk** from `name`. The namespace prefix registered for `module` is combined with `local_name` when a QName is required. It does not define semantic identity and MAY be adjusted in the **LHM reviewed** according to occurrence position and the rules in 5.3.7. Equality of `source_bsm_id` does not require equality of `local_name`.
18	xpath	The initial physical XPath generated mechanically from the ordered hierarchy by combining each row's `module` prefix with its initial `local_name`. **Post-Graph Walk** regenerates this value for every row.

5.3.6 Relationship among the three tables

FSM source	BSM result	candidate LHM
Authored Class or Abstract Class	Effective concrete `Class` row	Root or associated-Class occurrence with `type=C`
Authored or inherited Attribute	Effective `Attribute` row	Attribute occurrence with `type=A`
Specialisation Association	Resolved as inheritance and removed	Not traversed or emitted
Composition Association	Retained as an effective Association	Associated Class emitted and recursively traversed
Aggregation Association	Retained as an effective Association	Associated Class emitted and recursively traversed while retaining aggregation semantics through `source_bsm_id`
Reference Association	Retained as an effective Association	Reference occurrence emitted with `type=R`; only target Attributes marked `PK` are copied below it as `REF`
Multiplicity `0` or `0..0`	Applied as a deletion directive and removed	Never emitted
`representation_term`	Retained for an effective Attribute	Copied to `datatype`
`property_term`	Retained for an effective Attribute	Used in `name` and `semantic_path`
`association_role` and target Class	Retained on the effective Association	Used to construct the associated-Class occurrence name and semantic path
No generated row identifier	Unique **BSM** `id` generated	Copied to `source_bsm_id`

FSM source	BSM result	candidate LHM
No hierarchical semantic path	No hierarchical semantic path	`semantic_path` generated by **Graph Walk**
`module` and occurrence `name`	Retained semantic context	Initial `local_name` generated mechanically by **Graph Walk** from `name`
No physical XPath	No physical XPath	Initial `xpath` generated by **Graph Walk** from the occurrence hierarchy by combining each row's `module` prefix with its initial `local_name`

Each **LHM** row is identified by `semantic_path`; `source_bsm_id` provides traceability to the originating **BSM** row.

A conformant **LHM** MUST satisfy all of the following correspondence conditions:

- each `semantic_path` corresponds to exactly one `source_bsm_id`;
- a source_bsm_id MAY occur in multiple **LHM** rows when the originating **BSM** row is reached through different **Graph Walk** paths
- each semantic_path occurs exactly once; and
- the content associated with each `semantic_path` is consistent.

A duplicate semantic_path, an unresolved source_bsm_id, or conflicting content for the same semantic_path indicates an **FSM** or **BSM** design error. Reuse of one source_bsm_id on different unique semantic_path values is permitted and is not an error.

The candidate may be retained for diagnosis only; the error MUST be corrected and **Graph Walk** rerun before the **candidate LHM** may be semantically approved.

5.3.7 LHM review after Graph Walk

After a **candidate LHM** has passed the conformance checks in 13.4 and has been semantically approved, a separate copy MUST be prepared as the **LHM reviewed** before **post_graphwalk** processing. The **candidate LHM** MUST remain unchanged.

The **LHM reviewed** MUST contain exactly 18 columns. Columns 1–8 and 10–18 retain the same positions as the candidate LHM, while column 9 is `value_domain` instead of the candidate-LHM `association_role`. The reviewed column order is: `sequence,module,level,type,identifier,name,datatype,multiplicity,value_domain,definition,label_local,definition_local,source_bsm_id,semantic_path,associated_module,class_term,local_name,xpath`.

During preparation of the **LHM reviewed**, the responsible model definer MUST assign a stable `value_domain` identifier to each Attribute occurrence for which a governed Value Domain applies. The former `association_role` value MUST NOT be transferred into `value_domain`; the two fields have different meanings. Class and Reference rows normally have an empty `value_domain`, and an Attribute for which no governed Value Domain applies also has an empty value. A non-empty `value_domain` MUST resolve to exactly one authoritative Value Domain definition. The review MUST NOT infer a Value Domain solely from `name`, `datatype`, `local_name`, lexical examples, or the former `association_role`.

During preparation of the **LHM reviewed**, each Class, Attribute, and Reference occurrence MUST be assigned an appropriate binding-specific `local_name` according to its occurrence position. A `local_name` MAY be manually adjusted to resolve a duplicate (`module`, `local_name`) combination, to distinguish separate occurrence-specific taxonomy declarations, or to provide another appropriate binding-specific local name. The adjustment MUST preserve a valid XML Schema NCName. The QName used in XPath is constructed from the namespace prefix registered for `module` and the reviewed `local_name`.

Where multiple occurrences originate from the same **BSM** component, equality of `source_bsm_id` records their common semantic origin. Equality requirements for `local_name`, Class QName, Property QName, type QName, and directly contained Property QNames are determined independently for each occurrence path.

For example, occurrences derived from one underlying Document Class MAY be bound as `document`, `headerDocument`, and `detailDocument`, with occurrence-specific type names and child names such as `documentID`, `hDocumentID`, and `dDocumentID`. These are distinct taxonomy declarations even where their rows trace to the same underlying **BSM** model.

A reviewed change to `local_name` preserves `name` and `semantic_path`. The responsible model definer MUST NOT modify `name` merely to obtain taxonomy-name uniqueness; a pure syntax-name collision is resolved through a permitted `local_name` adjustment. A defect in `name` that reflects an error in the semantic model or **Graph Walk** naming rules requires correction of the applicable FSM source or processing rule, followed by rerunning Specialisation and **Graph Walk**.

The review MAY restrict `multiplicity` where required by the applicable business requirements, jurisdiction, regulation, or implementation scope. A reviewed multiplicity MUST be a subset of the occurrences permitted by the corresponding multiplicity in the **candidate LHM**.

Permitted restrictions include:

- making an optional occurrence mandatory, for example `0..1` → `1..1`;
- designating an optional occurrence as not used by changing its multiplicity to `0..0`;
- reducing the maximum number of occurrences, for example `0..*` → `0..1` or `1..*` → `1..1`.

A reviewed maximum occurrence MUST be at least the candidate-LHM minimum occurrence. A mandatory candidate occurrence remains in use.

A `0..0` multiplicity is a non-use designation. The corresponding row MUST remain in the **LHM reviewed** for traceability.

The **LHM reviewed** MUST preserve `sequence`, `module`, `level`, `type`, `identifier`, `name`, `datatype`, `definition`, `label_local`, `definition_local`, `source_bsm_id`, `semantic_path`, `associated_module`, `class_term`, `xpath`, row membership, and row order. Column 9 is the controlled replacement of candidate-LHM `association_role` by reviewed `value_domain`. Only that controlled Value Domain assignment, the permitted `local_name` adjustments, and the permitted `multiplicity` restrictions may differ from the semantically approved candidate LHM.

Consequently, the candidate LHM and the **LHM reviewed** MAY differ only in the controlled column-9 replacement and reviewed `value_domain` assignment, permitted `local_name` adjustments, and permitted `multiplicity` restrictions. Each review decision MUST be recorded as specified in 7.1. If any other value requires correction, the applicable FSM source MUST be corrected, and Specialisation and Graph Walk MUST be rerun.

5.3.8 Post-Graph Walk processing

post_graphwalk MUST accept an approved **LHM reviewed** and produce one formal **HMD_for_taxonomy** for each effective `level = 1` root. The **candidate LHM** and **LHM reviewed** remain unchanged.

For each row, post_graphwalk MUST preserve the reviewed `value_domain`, `local_name`, and `multiplicity`; preserve `semantic_path`; and regenerate `xpath` from the reviewed hierarchy, `module`, and `local_name`. `post_graphwalk` MUST NOT infer, add, replace, normalise, or otherwise decide a Value Domain.

An empty `local_name`, an invalid XML Schema NCName, or an unknown `module` is an error. The QName used in XPath MUST be constructed from the namespace prefix registered for `module` and the current `local_name`.

Before generation of the **HMD for taxonomy** output set, post_graphwalk MUST partition the **LHM reviewed** at each `level = 1` root and perform the `local_name` duplicate check separately within each resulting **HMD**. Within an HMD, the check MUST group the effective rows by (`module`, `local_name`).

The mechanical within-**HMD** check MUST detect duplicate (`module`, `local_name`) combinations and list every **LHM reviewed** row belonging to each duplicate group. The reported information MUST include at least the **HMD** root, `module`, `name`, `local_name`, `semantic_path`, and `source_bsm_id`.

The mechanical within-**HMD** check determines QName uniqueness from (`module`, `local_name`) and leaves semantic review decisions to the preceding LHM-review stage.

The responsible model definer MUST review every within-**HMD** duplicate group and adjust the applicable occurrence-specific `local_name` values in the **LHM reviewed** so that every (`module`, `local_name`) combination is unique within that **HMD**. The duplicate check MUST then be rerun.

The **HMD for taxonomy** conforms only when all within-**HMD** QName combinations are unique and all cross-**HMD** shared-QName definitions are consistent.

Validation scope is one **HMD for taxonomy** / one DTS. QName consistency and closure are assessed within the selected root-specific DTS. Concepts reused across different root-specific DTSs are compared only when they are deliberately governed as the same shared declaration; unrelated cross-DTS equality is not a conformance requirement.

The same (`module`, `local_name`) combination MAY occur in different **HMDs** where the taxonomy design deliberately reuses one shared declaration, including a declaration in a shared module such as `bus`, `muc`, `emh`, or `tax`. Each such cross-**HMD** reuse MUST have identical concept-defining values and one identical authoritative content model. A palette-specific occurrence that requires a different declaration MUST instead use an occurrence-specific `local_name` or the applicable palette-specific module.

Neither equality nor inequality of `source_bsm_id` determines whether a taxonomy declaration is shared. Shared-declaration validation is based on the reviewed QName and its concept-defining values; `source_bsm_id` remains a traceability identifier only.

After all within-**HMD** uniqueness checks and cross-**HMD** shared-declaration checks have passed, post_graphwalk MUST preserve each reviewed `local_name` and MUST regenerate `xpath` separately for every **HMD** from the ordered hierarchy by constructing each XPath step QName from the row's `module` prefix and current `local_name`.

A level-1 Class MUST be placed below `/xbrli:xbrl`. Each lower-level occurrence MUST be appended to the XPath of its immediate Class or Reference parent. Attribute rows are leaves. Regeneration MUST reflect a change to `module` or `local_name` in that row and every descendant row.

Within one **HMD**, an XPath assigned to different `semantic_path` values is an error.

For XBRL GL Next v0 (August 2026), no LHM or HMD for taxonomy contains a `domain_name` column. The candidate LHM retains `association_role` as column 9 because Graph Walk requires Association context to construct the occurrence hierarchy and `semantic_path`. The controlled LHM-review step replaces column 9 with `value_domain` and records the reviewed Value Domain assignment. Post-Graph Walk accepts that reviewed 18-column contract and preserves `value_domain` unchanged into each HMD for taxonomy. Association-role semantics remain traceable through the candidate LHM, `source_bsm_id`, and the unchanged `semantic_path`.

5.3.9 HMD for taxonomy definition table

The HMD for taxonomy MUST contain 18 columns. Columns 1–8 and 10–18 retain their established positions. Column 9 is `value_domain`, replacing the LHM/LHM-reviewed `association_role` column for taxonomy-generation input. This replacement preserves the canonical column count and position while separating Association semantics from taxonomy **Value Domain** binding.

No.	Column	Definition in the HMD for taxonomy
1	sequence	The unchanged **Graph Walk** output order.
2	module	The unchanged module value.
3	level	The unchanged hierarchy level.
4	type	The unchanged occurrence type: `C`, `A`, or `R`.
5	identifier	The unchanged identifier-role marker, including `PK` or `REF` where applicable.
6	name	The unchanged human-readable occurrence name generated by **Graph Walk**.
7	datatype	The unchanged semantic datatype or representation term.
8	multiplicity	The multiplicity preserved from the **LHM reviewed** for each effective output row. An occurrence whose reviewed multiplicity is `0..0`, together with its occurrence subtree, is excluded from the **HMD_for_taxonomy** files.
9	value_domain	Stable governed Value Domain identifier used by taxonomy generation for an Attribute, or empty where no Value Domain applies. It is assigned in the **LHM reviewed** and preserved unchanged by post_graphwalk into the HMD for taxonomy. It replaces candidate-LHM `association_role` at column 9 and does not alter `datatype` or `semantic_path`.
10	definition	The unchanged base-language definition.
11	label_local	The unchanged local-language label.
12	definition_local	The unchanged local-language definition.
13	source_bsm_id	The unchanged **BSM** row identifier retained for traceability.
14	semantic_path	The unchanged canonical semantic path generated by **Graph Walk**, validated but neither repaired nor regenerated by **Post-Graph Walk** processing, and used as the syntax-independent row identifier and binding anchor.
15	associated_module	The unchanged module of the Class context represented by the row.
16	class_term	The unchanged Class context of the occurrence.
17	local_name	The XML Schema NCName preserved from the **LHM reviewed**.
18	xpath	The physical XPath regenerated by post_graphwalk from the current hierarchy by combining each row's `module` prefix with its current `local_name`.

Each formal **HMD** MUST preserve the relative row order of the effective rows in the corresponding root-specific portion of the **LHM reviewed**. An occurrence whose reviewed multiplicity is `0..0`, together with every descendant row in its occurrence subtree, MUST be excluded.

post_graphwalk MUST validate each excluded `0..0` occurrence and its subtree as part of the complete **LHM reviewed** input. Excluded rows do not occur in a formal **HMD**.

The HMD for taxonomy MUST preserve `associated_module` and all other retained provenance fields. Column 9 is `value_domain`, and each value MUST be preserved unchanged from the corresponding approved **LHM reviewed** row. `post_graphwalk` MUST NOT infer or create a Value Domain assignment. Association-role semantics remain traceable from the candidate LHM through `source_bsm_id` and the unchanged `semantic_path`.

A change to `value_domain`, `local_name`, or `multiplicity` is made in a new revision of the **LHM reviewed**, after which post_graphwalk is rerun.

5.3.10 Relationship among candidate LHM, LHM reviewed, and HMD for taxonomy

Characteristic	candidate LHM	LHM reviewed	HMD for taxonomy
Purpose	Candidate hierarchy produced by **Graph Walk**; eligible for semantic-model approval after the checks in 13.4	Controlled human-review artefact	Deterministic formal input to a **conformant taxonomy generator**
Producer	**Graph Walk**	Responsible model definer	**Post-Graph Walk**
Lifecycle status	Generated artefact; may be semantically approved after the conformance checks in 13.4	Prepared from a semantically approved **candidate LHM**; controlled review artefact	Approved formal input to a **conformant taxonomy generator** after all **Post-Graph Walk** checks pass
Column count	18	18	18
Row scope	Complete **Graph Walk** result for the supplied roots	Exactly the same rows in the same order	Effective rows in the same relative order; each `0..0` occurrence subtree is excluded
name	Generated from **FSM/BSM** terms and Association role	Preserved unchanged	Preserved unchanged
multiplicity	Generated from the conformant **FSM source** and **BSM result**	MAY be restricted to a subset. An optional occurrence MAY be designated as not used by setting its multiplicity to 0..0.	Preserved from the **LHM reviewed** for effective rows. Each occurrence designated as `0..0`, together with its occurrence subtree, is excluded from the **HMD_for_taxonomy** files.
association_role / value_domain	`association_role` present	Column 9 is `value_domain`; reviewed Value Domain assignment is controlled and `association_role` is not copied	`value_domain` preserved unchanged from the **LHM reviewed**
associated_module	Present	Preserved unchanged	Preserved unchanged
source_bsm_id	Generated from **BSM** id	Preserved unchanged	Preserved unchanged
semantic_path	Generated by **Graph Walk**	Preserved unchanged; a path defect requires correction of the source or **Graph Walk** processing and regeneration of the **candidate LHM**	Preserved unchanged after canonical-form validation
local_name	Initial `local_name` generated mechanically from `name` by **Graph Walk**	MAY be adjusted only as permitted	Preserved unchanged from the **LHM reviewed**
xpath	Initial `xpath` generated by **Graph Walk** from the hierarchy, using each row's `module` prefix and initial `local_name`; provided for review and traceability	Copied unchanged from the **candidate LHM** as a provisional value. It may no longer correspond to an adjusted `local_name`; **Post-Graph Walk** regenerates it before formal use	Regenerated by **Post-Graph Walk** for every effective row from the hierarchy, using each row's `module` prefix and reviewed `local_name`

The **LHM reviewed** is a controlled review artefact prepared from a semantically approved candidate LHM. It preserves the approved semantic content while recording the controlled `value_domain` assignment, permitted `local_name` adjustments, and permitted `multiplicity` restrictions defined by this document.

A reviewed multiplicity MUST remain within the occurrence range permitted by the **candidate LHM**. Therefore, the **LHM reviewed** can impose a stricter requirement or designate an optional occurrence as not used, but it cannot introduce an occurrence that the **candidate LHM** does not permit.

5.3.11 Datatype binding and Value Domain generation

The semantic `datatype` in each formal HMD remains unchanged. For every Attribute, formal generation MUST resolve the XBRL item type by applying an explicit occurrence or concept override first and the approved default semantic-datatype mapping second. If neither resolves the datatype, or if the applicable mapping is `review-required`, generation MUST fail. A generator MUST NOT silently fall back to `xbrli:stringItemType` or infer an item type from `name`, `local_name`, or a name suffix.

Value Domain identity is assigned explicitly in the approved **LHM reviewed** `value_domain` column and preserved unchanged into the **HMD for taxonomy**. Each non-empty value MUST resolve against the authoritative Value Domain/Value inputs supplied to the **conformant Taxonomy Generator**. Datatype binding remains separate: `datatype` retains the semantic representation term, while `value_domain` identifies the governed permissible-value domain.

Value Domain generation is deterministic. Where an approved `value_domain` requires an extensible XBRL enumeration, the **conformant Taxonomy Generator** MUST generate Extensible Enumerations 1.0 domain-head and member concepts, the domain-member relationships and deterministic **Value Domain** linkrole, and the applicable concept binding from the authoritative **Value Domain**, Value, overlay, and datatype inputs. An EE1-bound fact reports the QName of one permitted member. Extensible Enumerations 2.0 multiple-value semantics are not required for Framework **Value Domain** facts. A **Value Domain** identifier that does not resolve, a Value Domain assigned to a non-Attribute row, or an applicable domain with no resolved values is a generation error.

5.3.12 Class hierarchy and occurrence identity

Class hierarchy is the semantic and structural nesting defined by the authoritative HMD. Occurrence identity is the identity of repeated instances in an instance-oriented representation. A Class MUST NOT acquire an occurrence key merely because it is a Class.

Occurrence-key classification MUST be derived deterministically from the effective HMD hierarchy and multiplicity. The root Class is an **occurrence-key Class**. A Class whose effective maximum occurrence is greater than one is an **occurrence-key Class**. A non-root Class with multiplicity `0..1` or `1..1` is not an **occurrence-key Class**. No additional HMD column, including an `occurrence_key` or `domain_name` column, is introduced.

Selector-effective multiplicity is a taxonomy-generation derivation, not a semantic-model edit. When two or more distinct selector-qualified occurrences resolve to the same canonical occurrence owner, the Generator MAY derive an effective repeated maximum for the generated OIM occurrence dimension and MUST record the source and effective multiplicities. It MUST NOT directly rewrite the accepted HMD semantic multiplicity (for example, by changing an accepted `0..1` Class to `0..*` merely because physical selector variants exist).

Structured CSV MUST define occurrence-key columns only for **occurrence-key Classes**. A non-repeated Class remains in the semantic hierarchy and is reconstructed from that hierarchy and the applicable binding, but it does not create an occurrence-key column.

```
Invoice [root]
  InvoiceLine [1..*]
    ItemInformation [1..1]
      ItemAttributes [0..*]
```

The required **Structured CSV** occurrence keys for this hierarchy are `dInvoice`, `dInvoiceLine`, and `dItemAttributes`. There is no `dItemInformation` occurrence key because `ItemInformation` is singular within each `InvoiceLine` occurrence.

For nested repeated Classes such as `Parent [0..*]` containing `Child [0..*]`, each Child occurrence identity includes the applicable Parent occurrence identity. Child ordinals or keys are therefore interpreted within their parent occurrence scope.

Value Domain classification is independent of occurrence-key classification. A scalar Attribute with an enumerated **Value Domain** remains a scalar fact and does not create an occurrence key or dimension merely because its permitted values are enumerated.

5.4 Modules and taxonomy assemblies

5.4.1 Module types

The framework supports modular taxonomy packaging comprising:

a) Core module: jurisdiction-neutral foundational concepts required for minimum interoperability;
b) Business document modules: order/purchase, shipment/logistics, invoicing, payments/settlement;
c) Accounting book modules: journal entries, ledger structures (GL/AP/AR), trial balance, and adjustments including cash application/payment;
d) Cross-cutting capability modules: multi-currency, segment classification, activity logging, reporting drill-down/roll-up mappings, statistical/non-financial measures;
e) Jurisdiction modules: localisation (labels, code lists, constraints, and mandated sets).

5.4.2 Use-case taxonomy assemblies

A use-case taxonomy assembly is a named syntax realisation generated from an **HMD for taxonomy** under the fixed rules of this Framework and the applicable binding Part. Its semantic content is traceable through the **FSM**, **BSM**, and **LHM**.

The applicable roots are represented by the root-specific **HMDs** produced from the approved **LHM reviewed**. Module, namespace, binding, presentation, message-entry-point, and validation behaviour are determined by the LHM artefacts, module definitions, and fixed Framework rules rather than by a separate run-specific declaration.

5.4.3 Semantic and syntax bindings

The framework supports:

a) semantic binding to controlled vocabularies and code lists (including tax category/type schemes and other classifiers);
b) syntax binding to the serialisations defined in Part 2 and Part 3.

Implementations supporting both XBRL XML and OIM serialisations provide deterministic transformation without semantic loss under the applicable fixed binding rules.

5.4.4 `gen` module and Value Domains

The `gen` semantic module MUST govern binding-neutral reusable semantic definitions. Its intended scope includes reusable basic semantic types, governed legacy, standard, and user-defined **Value Domain**s, and other reusable semantic definitions needed by multiple business or accounting modules. An **FSM** MAY govern those definitions and their semantic ownership.

The **HMD** `datatype` is a semantic representation term, not an XBRL item type. XBRL datatype binding is a separate explicit operation: an approved occurrence or concept override takes precedence over the approved default mapping, and an unresolved or `review-required` datatype is a generation error. Datatype binding MUST NOT infer a type from an Attribute name, `local_name`, lexical value, or suffix.

Confirmed bindings include `Amount` to `xbrli:monetaryItemType`, `Percentage` to `xbrli:pureItemType`, `Quantity` to `xbrli:decimalItemType`, `Code` to `xbrli:tokenItemType`, `Text` and `Identifier` to `xbrli:stringItemType`, and `Binary object` through `xs:base64Binary` to `xbrli:base64BinaryItemType`. The Binary object binding is a datatype example and does not define a **Value Domain**.

A **Value Domain** identifies the governed permitted semantic value set and is independent of datatype. For example, `Code` paired with `UNCL 1001 Document Name Code` and `Code` paired with `XBRL GL Legacy Document Type` share a general semantic datatype but have different **Value Domain** identities. Neither Attribute names nor lexical forms determine **Value Domain** identity, and **Value Domain** binding does not alter the **HMD** semantic datatype.

Legacy, standard, and user-defined **Value Domains** MUST retain separate identities. The 18 XBRL GL 2003-compatible legacy code and value domains MUST remain available for backward compatibility and MUST NOT be silently replaced by related standard vocabularies. A standard **Value Domain** MUST identify its external authority, vocabulary identifier, edition or version, applicable subset, provenance, and rights or redistribution status where applicable. Reference to an external code list does not grant redistribution rights. A user-defined **Value Domain** MUST retain its own governed identity even when some lexical values match a standard **Value Domain**.

A **Value Domain** crosswalk MUST preserve both source and target identities and SHOULD classify correspondences as `exact`, `equivalent`, `partial`, `no-equivalent`, or `legacy-only`, as applicable. Normal reporting and validation MUST NOT silently translate legacy values. Translation through a crosswalk is an explicit transformation operation.

The semantic **Value Domain** identity is distinct from a physical external code-list, schema, or distribution artefact. A standard **Value Domain** MAY reference an authoritative external code list instead of reproducing every value in the **FSM**. Provenance and third-party ownership MUST be preserved.

The one shared generated `gen` taxonomy module MUST be used by both the Tuple and OIM bindings. Tuple-specific content structures and OIM-specific dimensional structures MUST remain in their respective binding modules and MUST NOT be introduced into `gen`; `gen` is not merely a collection of XSD helper types.

Where an approved binding requires an extensible XBRL enumeration, EE 1.0 is the required Framework realisation so that one fact reports one **Value Domain** member. Extensible Enumerations 2.0 multiple-value semantics are not required for Framework **Value Domain** facts. The **conformant Taxonomy Generator** deterministically generates the domain/member relationships and concept bindings from authoritative `value_domain`, **Value Domain**, Value, overlay, and datatype inputs.

The 18-column candidate LHM retains `association_role` in column 9. The controlled 18-column LHM-reviewed contract replaces that column in-place with `value_domain`, and the HMD for taxonomy preserves the reviewed `value_domain` unchanged. No `domain_name` column is introduced. `value_domain` is the formal taxonomy-generation binding and MUST NOT change `semantic_path`, `datatype`, or occurrence identity.

5.5 XBRL QName and concept naming

5.5.1 XBRL concepts and QName

A QName combines a namespace URI and a local name. XBRL items and tuples are global element declarations identified by QName. The **BSM** `id`, LHM `source_bsm_id`, and semantic-model identifiers serve different, syntax-independent purposes. In the LHM, `semantic_path` provides semantic occurrence identity, `source_bsm_id` provides semantic-source traceability, `name` is the human-readable semantic occurrence name, and `local_name` is the taxonomy implementation name. The taxonomy QName is determined by the namespace registered for `module` together with `local_name`.

Within one root-specific **HMD for taxonomy**, each (`module`, `local_name`) combination MUST identify exactly one reviewed LHM occurrence and `semantic_path`. The same combination MAY occur in different **HMDs** only when it deliberately identifies one shared taxonomy declaration whose concept-defining values and authoritative content model are identical in every **HMD**. Ordinary XML Schema local elements can instead be distinguished by their containing `complexType`.

In an XBRL Tuple taxonomy, a tuple `complexType` places global XBRL items or tuples by `ref`.

5.5.2 Prohibited and corrected example

A detail amount and a semantically different header amount are represented by separate **FSM** and **BSM** rows and bind to distinct QNames, for example `gl-cor:amount` and `gl-cor:headerAmount`.

Graph Walk derives the initial `local_name` mechanically from `name`; `module` supplies the namespace context used to construct the QName. A pure syntax-name collision is resolved by adjusting `local_name` in the **LHM reviewed**. A conflict that reveals a semantic-model error is corrected in the applicable **FSM source**, after which **Specialisation** and **Graph Walk** are rerun.

5.5.3 `local_name` reuse and duplicate checks

After a **candidate LHM** has been semantically approved, an **LHM reviewed** MUST be prepared in accordance with 5.3.7.

The `local_name` duplicate check MUST use the current reviewed `local_name` and perform grouping by (`module`, `local_name`) separately within each root-specific **HMD**.

The check MUST determine the within-HMD duplicate groups and compare the concept-defining values of each QName used in more than one HMD. It uses `source_bsm_id` only for traceability and leaves permitted `local_name` selection to the LHM-review stage.

The responsible model definer MUST assign occurrence-specific `local_name` values so that every (`module`, `local_name`) combination is unique within each HMD. A repeated QName across different HMDs MAY remain only as a deliberate shared declaration with identical concept-defining values and one authoritative content model; otherwise, the applicable occurrence MUST be assigned a different `local_name` or assigned to the appropriate palette-specific module.

The checks MUST be rerun after each adjustment. post_graphwalk MUST proceed only when no within-**HMD** duplicate remains and every cross-**HMD** shared-QName comparison is consistent.

Initial duplicate (`module`, `local_name`) combinations are not semantic-model errors and do not prevent semantic-model approval of the **candidate LHM**. They MUST be resolved in the **LHM reviewed** before **Post-Graph Walk** produces the **HMD for taxonomy**. A duplicate `name` is not, by itself, a taxonomy conformance failure. A duplicate `semantic_path` is an LHM semantic-occurrence identity error. A duplicate (`module`, `local_name`) combination within one HMD is a taxonomy QName collision requiring a permitted `local_name` adjustment in the **LHM reviewed**.

5.6 Semantic Path

5.6.1 Purpose and Reference

The **Semantic Path** is a deterministic Framework-specific string used to identify and locate a semantic element within the Logical Hierarchical Model (**LHM**). It represents the hierarchy and relationships produced by **Graph Walk** and serves as the authoritative common anchor for syntax bindings. It is not declared to be JSONPath.

5.6.2 Syntax Rules and Construction

The **Semantic Path** MUST be constructed from the deterministic **Graph Walk** result using the following rules:

- Root marker (`$`): Every path begins with `$`, followed by the root Class segment.

- Segment separator (`.`): A full stop separates successive Class, associated-Class, and Attribute segments.
- Term normalisation: For every term used in a segment, every character other than the ASCII letters `A` to `Z` and `a` to `z` MUST be removed. This removes whitespace, underscores, hyphens, slashes, commas, full stops, parentheses, digits, and all other non-ASCII-letter characters. The relative order and case of the retained letters MUST remain unchanged. A term that normalises to the empty string is invalid.
- Segment construction:
- Class segment: `module + "_" + normalised(class_term)`;

  where `module` is the LHM `module` value, not a taxonomy namespace prefix; for example, `cor_AccountingEntries`.

  Attribute segment: `module + "_" + normalised(property_term)`.

  Associated-Class segment: `associated_module + "_" + normalised(association_role + associated_class)`;

  `association_role` and `associated_class` MUST be concatenated without an intervening separator before normalisation; when `association_role` is empty, only the associated Class term follows the underscore.

- Path determinism: The path MUST record the ordered Class and property occurrences produced for the selected root.

  Example: `$.cor_AccountingEntries.cor_EntryHeader.cor_EntryDetail.cor_DebitCreditIndicator`.

5.6.3 Functional Role in the Framework

The **Semantic Path** records the hierarchy followed by **Graph Walk** and is the common reference used by later syntax bindings. It is not an input for automatically inventing a unique XBRL concept name.

- Binding comparison: A syntax binding records which reviewed XBRL concept QName, XPath, JSON expression, or CSV column corresponds to an LHM `semantic_path`.
- Path preservation: Any syntax-specific paths added later preserve the LHM `semantic_path` unchanged.
- Reference handling: A Reference Association path terminates at the copied `identifier=PK` row or at the reference row when no PK is available.

5.6.4 Conformance and Validation

Semantic Path conformance MUST be verified by confirming the following conditions:

- The **Semantic Path** preserves the exact sequence of Composition, Aggregation, and Reference Association occurrences traversed or emitted during **Graph Walk**.
- Every segment uses the applicable LHM `module` value followed by one underscore and a non-empty normalised term matching `[A-Za-z]+`.
- Each `semantic_path` occurs exactly once in the **candidate LHM** and identifies one row content. Distinct source terms that normalise to the same segment under the same parent path and module constitute a path collision.
- **Graph Walk** MUST reject a term that normalises to the empty string and MUST report a duplicate or conflicting complete path as `SEMANTIC_PATH_CONFLICT`. **Post-Graph Walk** processing MUST reject a non-canonical `semantic_path` without repairing or regenerating it.

6 Extension Mechanisms and Governance

6.1 Semantic authority and extension point

Semantic extensions are defined in an **FSM** file. The resulting **BSM**, **LHM**, and syntax-binding artefacts preserve traceability to those **FSM** definitions.

An extension can add a Class or property, specialise an existing Class, change an Association type, restrict a value domain or multiplicity, apply a deletion directive, or declare a code-list dependency.

6.2 Shared, Aligned, and Distinct

Shared, Aligned, and Distinct classify semantic dependency and reuse scope, not merely folder placement or origin.

- Shared definitions are suitable for broad reuse across countries, institutions, industries, and implementations.
- Aligned definitions specialise Shared definitions while preserving dependence on a stated public business context, process, document structure, law, regulation, national or regional standard, or published industry standard.
- Distinct definitions are specific to an enterprise, corporate group, product, bilateral trading relationship, or closed community.

Classification as Shared is based on meaning, datatype, constraints, business context, and reuse scope. Origin in an international standard is supporting provenance rather than the sole classification criterion.

6.3 Namespace, ownership, and versioning

Each Shared, Aligned, or Distinct module MUST use a namespace under its maintainer's authority and record version, status, deprecation, replacement, and base-concept relationships.

6.4 Property matching, duplicates, and conflicts

Property matching and final-**BSM** duplicate checking use the semantic-model identifiers defined in Clause 3 and represented by the columns stated in 5.3.1. Same-content repetition and differing-content conflicts are reported separately.

6.5 Aligned Pool governance

An Aligned definition intended for reuse across jurisdictions or programmes MUST be registered in the Aligned Pool in accordance with Part 4.

Before introducing a new Aligned definition, the proposer MUST check the Shared definitions and existing Aligned Pool entries for semantic equivalence or overlap. The aligned extension registry records governance and lifecycle metadata; the aligned semantic library contains the accepted **FSM** definitions.

Accepted definitions MUST have stable semantic-model identifiers, a stated public alignment source, a maintainer, a version, and a lifecycle status. Inclusion in an **FSM** does not modify the authoritative Shared definition. Part 4 specifies submission, review, publication, versioning, deprecation, withdrawal, and licensing or sharing procedures.

7 Specialisation and Graph Walk

7.1 Scope, terminology, and processing stages

This Clause defines the required transformations and artefact conditions for **Specialisation** and **Graph Walk**. Conforming implementations MUST satisfy the processing requirements stated in this Clause. Programming language, command interfaces, variable names, storage structures, and diagnostic file formats remain implementation-specific.

Specialisation is the deterministic **FSM**-to-**BSM** process that resolves only Specialisation Associations and applies inheritance, additions, overrides, multiplicity constraints, and deletion directives.

A Specialisation Association is an Association whose association type is Specialisation. In the **FSM** file contract, this type is represented by the value `Specialisation` in `property_type`.

Composition, Aggregation, and Reference Associations remain in the **BSM** for **Graph Walk**.

Graph Walk is the deterministic depth-first **BSM**-to-**LHM** transformation from supplied root Classes.

Specialisation uses a conformant **FSM source**. **Graph Walk** uses a conformant **BSM result** and supplied root Classes. After semantic-model approval and preparation of the **LHM reviewed**, **Post-Graph Walk** uses that approved **LHM reviewed** as its authoritative input and produces one formal **HMD_for_taxonomy** for each effective level = 1 root. A **conformant taxonomy generator** subsequently uses one or more formal **HMDs** under the applicable binding Part.

Conformance is determined from the **FSM**, **BSM**, and **LHM** artefacts, the generated syntax-binding artefacts, and the observable relationships among them.

The artefacts and processing stages are shown in **Figure 2**. In the figure, rectangles represent persisted file artefacts, rounded rectangles represent script processing, ellipses represent manual processing, and the diamond represents a review decision. The figure is a lifecycle overview and does not imply that post_graphwalk receives the upstream **FSM source**, **BSM result**, or **candidate LHM** as runtime inputs.

Classes, Attributes, and Associations are identified by their respective semantic-model identifiers defined in Clause 3.

After **Graph Walk** has produced a **candidate LHM**, the responsible model definer MUST review it and assess its semantic-model conformance with the applicable requirements. The review, semantic-model approval, permitted adjustments, **post-Graph Walk** processing, confirmation, **HMD**-specific output, and taxonomy completion paths are shown in **Figure 2**. Candidate-to-reviewed comparison and approval are completed before post_graphwalk is run.

Graph Walk MUST generate the initial `local_name` and `xpath` values for every row of the **candidate LHM**. The **candidate LHM** remains unchanged after generation.

The semantic-model review MUST confirm that the **candidate LHM** correctly and completely represents the intended semantic model, including its occurrences, hierarchy, order, **semantic paths**, source **BSM** references, representation terms, datatypes, and multiplicities. Duplicate combinations of `module` and `local_name` do not constitute a semantic-model error and do not prevent semantic-model approval.

If the review or conformance assessment identifies a semantic-model error, the responsible model definer MUST correct the applicable **FSM** source and rerun Specialisation and **Graph Walk**.

Where the **FSM source** is corrected, **Specialisation** MUST produce a new **BSM result**, **Graph Walk** MUST produce a new **candidate LHM**, and the complete semantic-model review and conformance assessment MUST be repeated. Generated **BSM results** and **candidate LHMs** remain unchanged.

A **candidate LHM** MAY be semantically approved only after:

a) the **responsible model definer** has confirmed that it correctly and completely represents the intended semantic model;
b) all applicable **semantic paths**, source **BSM** references, occurrence relationships, hierarchy, order, representation terms, datatypes, and semantic-model multiplicities have been confirmed;
c) it has passed all applicable semantic-model and structural conformance checks; and
d) it has not been manually modified.

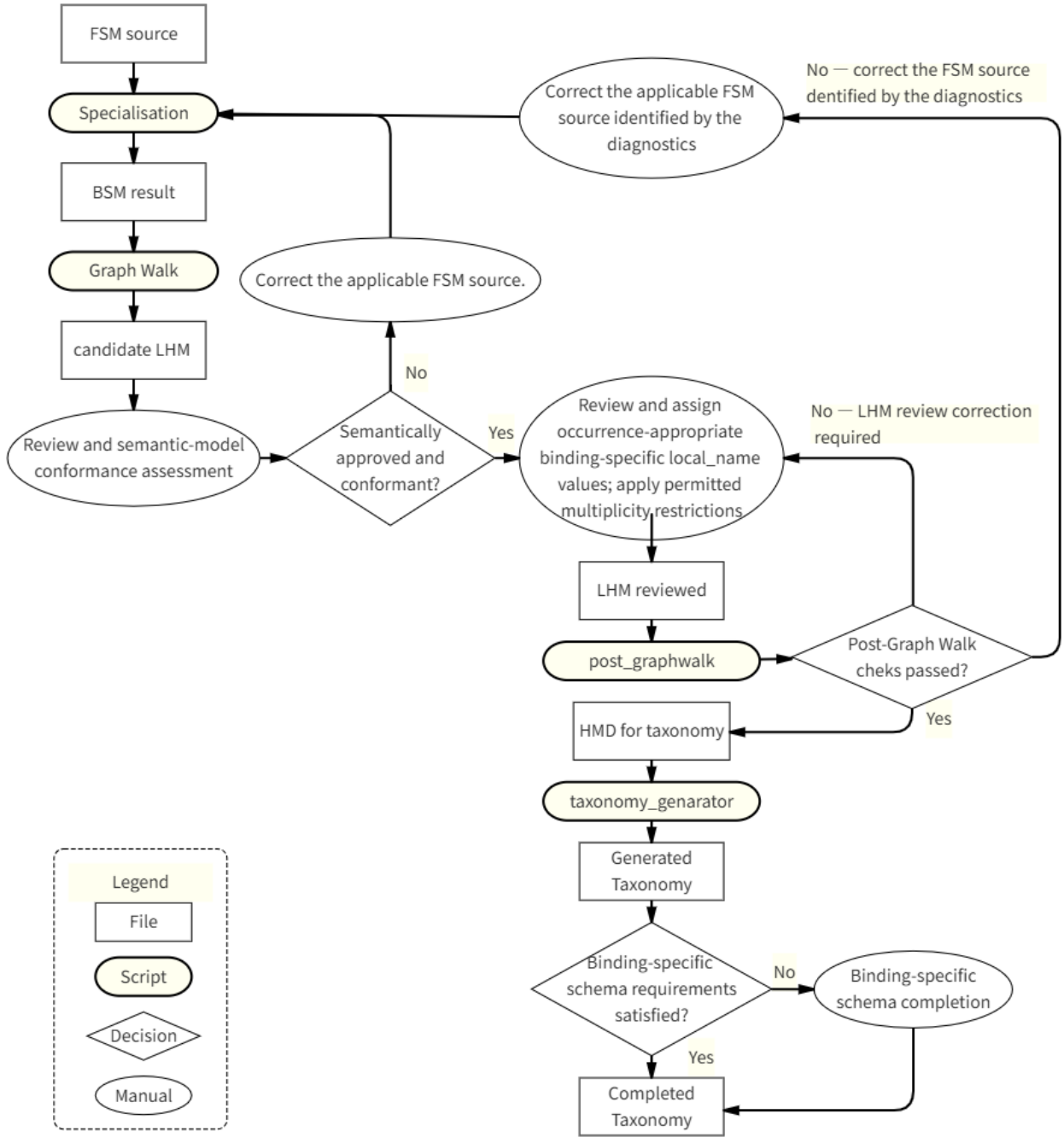


Figure 2 —Semantic-model review and taxonomy-generation workflow

After semantic-model approval, an **LHM reviewed** MUST be prepared from the candidate LHM as a controlled review artefact. The candidate LHM remains unchanged. During preparation of the **LHM reviewed**, the responsible model definer MAY make only the controlled changes specified below:

a) a `local_name` value, where necessary to assign an appropriate binding-specific name according to occurrence position, distinguish an occurrence-specific taxonomy declaration, or resolve a duplicate (`module`, `local_name`) combination within an HMD;;
b) a `multiplicity` value, where a restriction is required by the applicable business requirements, jurisdiction, regulation, or implementation scope and is expressly permitted by this document; and column 9 MUST be changed from **candidate LHM** `association_role` to `value_domain`, and a reviewed Value Domain identifier MUST be assigned only where a governed **Value Domain** applies.

A permitted `multiplicity` change MUST restrict the broader multiplicity of the approved semantic model for the intended business use.

All values other than the controlled column-9 replacement and reviewed `value_domain` assignment, permitted `local_name` adjustments, and permitted `multiplicity` restrictions MUST remain identical to those in the **candidate LHM**, including `name`, `sequence`, `semantic_path`, `source_bsm_id`, and the provisional `xpath` value retained for review.

The same **BSM** Class or Property MAY occur through more than one **Graph Walk** path. Such occurrences MAY have different reviewed `local_name` values and QNames. QName and child-QName equality is determined from the occurrence definitions, not solely from `source_bsm_id`.

For purposes of **Figure 2**, the box labelled `**HMD for taxonomy**` represents the set of formal **HMD**_for_taxonomy files produced for the effective `level = 1` roots. It does not represent a single combined output file.

NOTE — In Figure 2, the LHM-review activity includes the controlled replacement of candidate-LHM column 9 `association_role` by reviewed `value_domain` and the assignment of applicable reviewed Value Domain identifiers. The diagram does not imply that `post_graphwalk` determines a Value Domain.

The arrow from **LHM reviewed** to post_graphwalk represents its authoritative runtime data input. The upstream **FSM source**, **BSM** result, and **candidate LHM** are shown only as provenance and earlier lifecycle stages.

post_graphwalk MUST process the complete **LHM reviewed** and MUST:

a) validate the 18-column contract, sequence and level values, the requirement that the first data row has `level = 1`, and the hierarchy within each **HMD**;
b) treat every `level = 1` row as an **HMD** root, assign the following rows to that **HMD** up to the row before the next `level = 1` row, and assign the last **HMD** through the end of the file without changing row order;
c) include `multiplicity = 0..0` rows and their occurrence subtrees in input and structural validation and exclude each such subtree from every formal **HMD**_for_taxonomy;
d) preserve the reviewed `local_name`, `multiplicity`, and every other output value except `xpath`, and regenerate `xpath` separately for each **HMD** from that **HMD's** `level = 1` root by constructing each XPath step QName from the row's `module` prefix and current `local_name`;
e) perform semantic-path, regenerated-XPath, and QName uniqueness checks separately within each **HMD**, using the effective rows after exclusion of `0..0` rows; and
f) where the same QName occurs in different **HMDs**, determine consistency from the reviewed QName and concept-defining values rather than from `source_bsm_id`, verify that the authoritative declaration and content model are identical, report consistent reuse as `**SHARED_QNAME_ACROSS_HMDS**`, and report inconsistent definitions as `**INCONSISTENT_SHARED_QNAME_DEFINITION**` without automatically renaming the QName.

Post-Graph Walk performs only checks that can be determined from the approved **LHM reviewed**. Review-scope and business-justification decisions belong to the preceding LHM-review and approval stage.

The **HMD for taxonomy** conforms only when every formal **HMD** and every cross-**HMD** shared-QName comparison satisfies the applicable checks.

If a **Post-Graph Walk** check identifies a condition within the permitted review scope, the responsible model definer MAY prepare a new revision of the **LHM reviewed** and rerun **Post-Graph Walk**. A semantic-model error is corrected in the applicable **FSM source,** and the subsequent processing stages are rerun.

When all **HMD** and cross-**HMD** checks have passed, **Post-Graph Walk** produces the complete set of formal **HMDs**.

The approved revision of the **LHM reviewed** and every corresponding formal **HMD** MUST be preserved unchanged after approval. A required change is made in a new **LHM reviewed** revision and followed by the complete **Post-Graph Walk** and conformance process.

A **conformant taxonomy generator** MUST process one or more formal **HMDs** and produce an **HMD**-specific taxonomy entry point for each. A common module schema used by more than one **HMD** MUST be shared by those entry points. Palette-specific declarations remain in their owning module namespaces.

After taxonomy generation, each generated taxonomy entry point MUST be assessed to determine whether it satisfies all applicable binding-specific schema requirements. If it satisfies those requirements, it MAY proceed directly to the completed taxonomy. If additional binding-specific declarations or restrictions are required, binding-specific schema completion MUST be performed manually.

After any required schema completion has been performed, the completed taxonomy MUST be preserved as a separately identifiable release artefact. Where no manual schema completion is required, the generated taxonomy and the completed taxonomy MAY contain identical files but have different lifecycle statuses.

7.2 Specialisation: FSM to BSM

7.2.1 Inputs and registration

Specialisation accepts one or more conformant **FSM** files in their supplied order.

Whitespace handling preserves the semantic tokens represented in the **FSM** columns.

Class and property declarations are resolved by semantic-model identifier. Compatible declarations contribute to one effective Class; conflicting declarations are reported.

The **FSM** row types and Association types are the values defined by the 15-column contract.

A Specialisation Association MUST identify the specialising Class and its superclass by Class semantic-model identifier. Each superclass reference MUST resolve to exactly one **FSM** Class.

7.2.2 Semantic identifier and Class-reference resolution

A Class reference MUST resolve by its Class semantic-model identifier. The target of an Association matches (`associated_module`, `associated_class`) to (`module`, `class_term`).

A Class-term-only reference is conformant when it matches exactly one Class. Zero or multiple matches are reported as input errors.

7.2.3 Inheritance resolution

Specialisation MUST resolve every declared superclass before the specialising Class. A repeated Class in the active inheritance path MUST be reported as a Specialisation cycle, including the cycle path.

Inherited properties MUST be combined before local property directives are applied. When several **FSM** files contribute compatible declarations, their supplied order MUST be used deterministically.

Inherited items MUST be matched by semantic-model identifier. Compatible items MUST be merged using the least restrictive inherited multiplicity. A conflicting Attribute `representation_term` MUST be resolved by a local directive or reported as a conflict.

A conformant Association has an empty `representation_term`.

The resulting **BSM** retains Composition, Aggregation, and Reference Associations.

7.2.3.1 Cycle validation

Specialisation MUST identify a cycle when a Class semantic-model identifier recurs in its active inheritance path and MUST report the complete cycle path.

A successfully resolved Class MUST contribute its complete effective property set. A failed resolution contributes no partial Class result to a conformant **BSM**.

7.2.3.2 Resolution flow

Specialisation MUST follow only Specialisation Associations for inheritance. Composition, Aggregation, and Reference Associations MUST be validated and retained as effective **BSM** Associations.

Specialised classes are designed to meet diverse needs by building on established structures. For example, the class “Address” can be divided into “Physical Address” and “Billing Address.” In this case, “Physical Address” is a subclass of the “Address” class, inheriting attributes from the superclass while also having its own unique attributes.

To modify Class properties, including changes, additions, or removals, the following methodologies can be adopted:

a) Changing Multiplicity:

Adjusting the multiplicity of properties to better represent the variety and extent of relationships.

Abstract class multiplicity	Allowed specialised subclass multiplicity
0..1	0..1 0..* 1 1..*
0..*	0..1 0..* 1 1..*
1	0..1 0..* 1 1..*
1..*	0..1 0..* 1 1..*

b) Adding New Properties:

Introducing new properties with specific multiplicities to enhance subclass relevance and specificity..

c) Deselecting inherited properties by setting multiplicity to 0:

Setting inherited property multiplicities to 0 to effectively disable them when not applicable.

Abstract class multiplicity	Allowed specialised subclass multiplicity
0..1 0..* 1 1..*	0

These methods provide flexibility to accommodate various regional or sector-specific scenarios while preserving structural consistency.

Figure 3 illustrates implementation of Specialisation. Conformance is assessed from the resulting **BSM** and the requirements in this Clause.

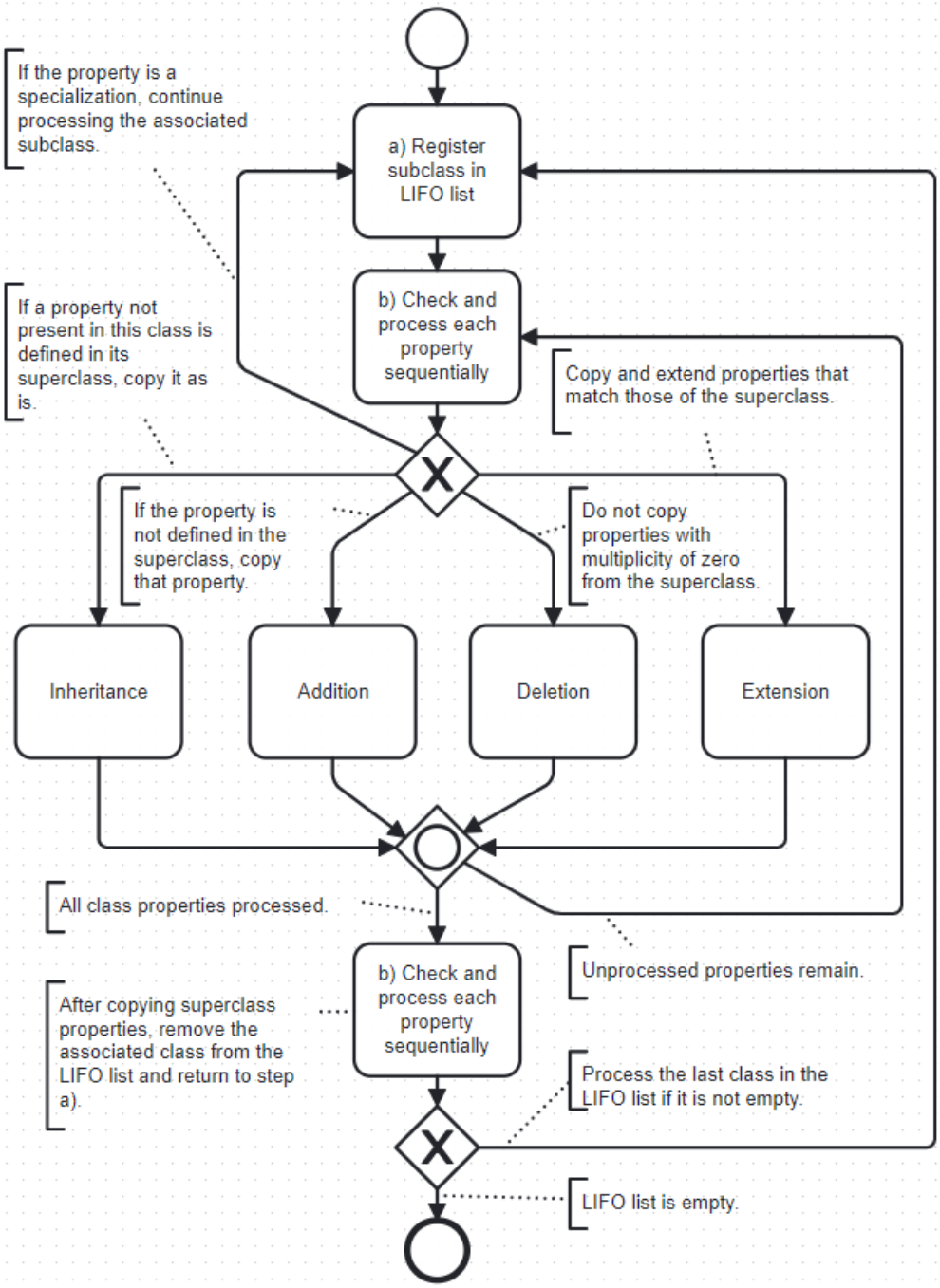


Figure 3 — Specialisation flow

A Specialisation Association establishes inheritance from the associated superclass to the specialising Class. Specialisation MUST process that inheritance independently of **Graph Walk**.

Specialisation processing:

For each specialising Class, Specialisation MUST resolve all of its superclasses before applying the Class's local property directives.

Specialisation MUST copy the resolved inherited property set, then apply local additions, overrides, multiplicity constraints, and deletion directives. Only Specialisation Associations are resolved during this process; Association-type changes are applied as content overrides.

Inherited Attributes MUST be matched by the Attribute semantic-model identifier: (`module`, `class_term`, `property_term`).

i. Inherited Associations MUST be matched by the Association semantic-model identifier: (`module`, `class_term`, `association_role`, `associated_module`, `associated_class`).
ii. A local Attribute with the same Attribute semantic-model identifier MUST override the inherited Attribute according to the local non-empty content fields.
iii. A local Association with the same Association semantic-model identifier can refine Association type or multiplicity. The Association role remains part of the identifier and is therefore unchanged by such an override.
iv. An Association role change MUST be expressed as two directives: a deletion directive for the Association semantic-model identifier containing the former role, and a new Association declaration with the Association semantic-model identifier containing the new role.
v. Multiplicity `0` or `0..0` MUST delete the matching inherited property. A local non-deletion property without an inherited match MUST be added.

7.2.4 Property matching and duplicate-check columns

Inherited-property matching and final-**BSM** duplicate checking MUST use the applicable semantic-model identifier.

Association type, `representation_term`, and multiplicity are content fields. Overrides and deletion directives are resolved before final-**BSM** validation; differing content for one semantic-model identifier is reported as a conflict.

7.2.5 Override, addition, and deletion

A local directive with the same semantic-model identifier MUST update the inherited item using its non-empty fields. A non-deletion directive without an inherited match MUST add an item.

Multiplicity `0` or `0..0` MUST remove the matching inherited property. An unmatched deletion MUST be reported as a validation result.

7.2.6 Abstract Classes and invalid Associations

Abstract Classes provide inheritance. A conformant **BSM** MUST contain concrete Classes and Associations whose target Classes resolve to concrete **BSM** Classes.

7.2.7 BSM output contract

The **BSM** MUST use these 16 columns in this order: `sequence`, `level`, `property_type`, `identifier`, `module`, `class_term`, `property_term`, `association_role`, `representation_term`, `associated_module`, `associated_class`, `multiplicity`, `definition`, `label_local`, `definition_local`, and `id`.

Concrete Class rows have level 1 and effective property rows have level 2. Local labels and definitions are preserved. Each **BSM** row has its assigned `id` and its semantic-model identifier.

7.2.8 Specialisation conformance

A conformant **BSM** MUST contain effective concrete Class, Attribute, Composition, Aggregation, and Reference rows with resolved Class references, unique semantic-model identifiers, and consistent content. Its `id` values provide unique row traceability.

7.3 Graph Walk: BSM to LHM

7.3.1 Inputs and root selection

Graph Walk MUST accept a conformant **BSM** and one or more root Classes supplied by Class semantic-model identifier. Each root MUST resolve to exactly one concrete **BSM** Class, and the supplied root order MUST determine output order.

Multiple roots can be represented in one **LHM** or separate **LHMs** when their order and partitioning are recorded.

7.3.2 Class map and property order

Graph Walk MUST resolve Classes and properties by semantic-model identifier and MUST preserve stable **BSM** property order. Duplicate or conflicting **BSM** items MUST be reported before a conformant **LHM** is produced.

7.3.3 Depth-first traversal

Graph Walk MUST produce a depth-first hierarchy using an abstract LIFO list. For each supplied root, it MUST PUSH the root Class, emit the root occurrence and its Attributes, and process its eligible Associations. After a child hierarchy is complete, it MUST POP the child Class and resume with the next property of the source Class.

Association selection SHOULD group Associations as mandatory singular (`1` or `1..1`), optional singular (`0..1`), and plural (`0..*` or `1..*`), preserving **BSM** order within each group. Another documented deterministic selection order can be used without making the result non-conformant.

7.3.3.1 Traversal state

An implementation can retain the current Class, occurrence segment, incoming Association, multiplicity, **semantic path**, ordered property list, and next-property position as traversal state.

For a non-root occurrence, the target Class supplies its effective properties and the incoming Association supplies the occurrence role, type, and multiplicity.

Figure 4 illustrates one depth-first implementation. Conformance is assessed from the resulting LHM and the requirements in this Clause.

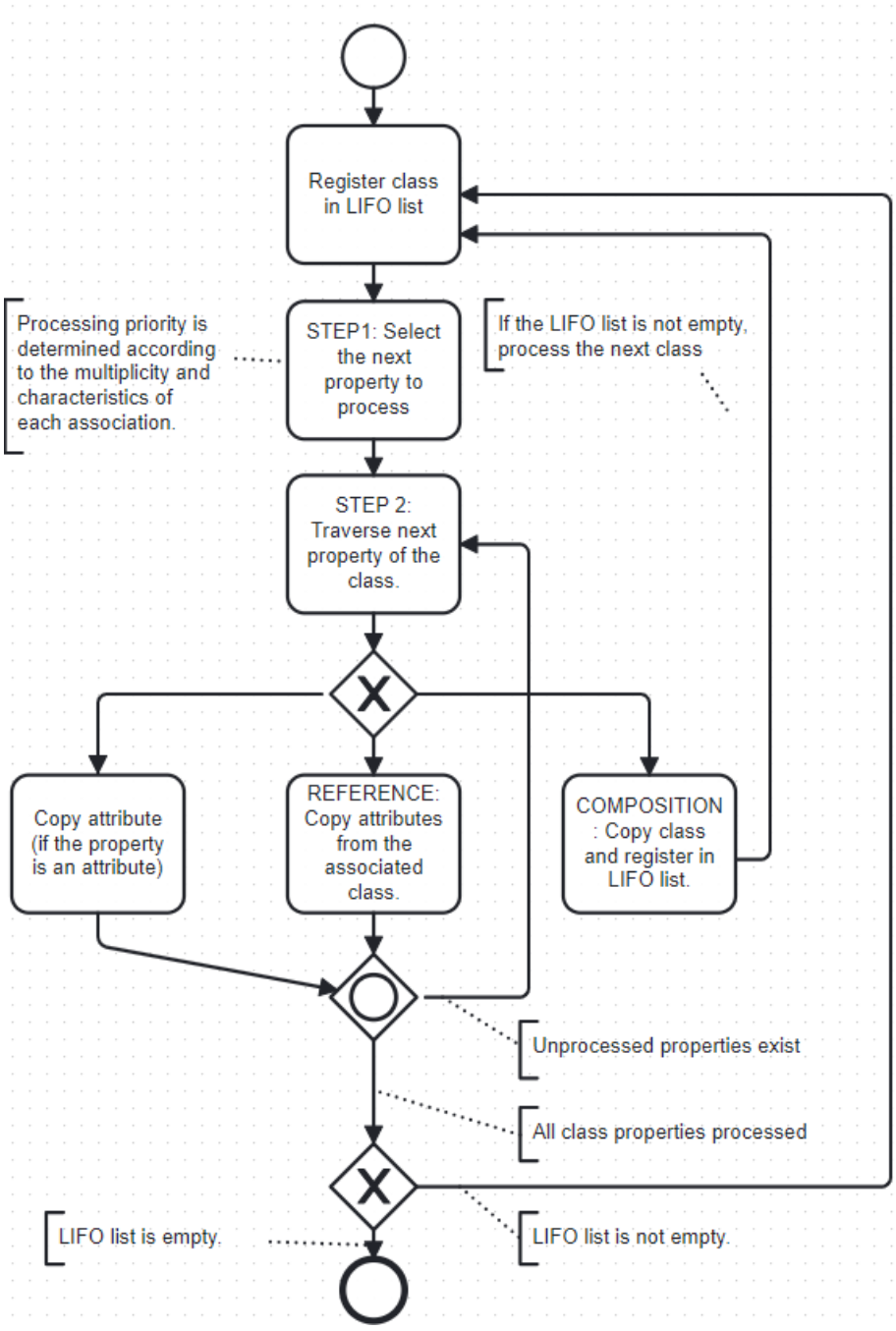


Figure 4 — Graph Walk flow

7.3.3.2 Traversal flow

Graph Walk MUST process every supplied root by depth-first traversal with last-in, first-out behaviour. The LIFO list defines the required processing order; an implementation may use recursion or another mechanism only when its observable PUSH, POP, traversal, and output order is equivalent.

For each root, **Graph Walk** MUST PUSH the root Class onto the LIFO list, place its occurrence in the **candidate LHM**, make it the current Class, and place it first in the active traversal path.

For the current Class, **Graph Walk** MUST emit its Attributes in stable **BSM** order and select each remaining Association according to 7.3.3.

For a Reference Association, **Graph Walk** MUST emit an **LHM** row with `type=R` at one level below the current Class and continue from the current traversal frame without recursively traversing the associated Class.

Graph Walk MUST then emit each target Class Attribute marked `identifier=PK` at one level below the `R` row, mark the emitted Attribute as `REF`, and retain the target Attribute's `source_bsm_id`.

For a Composition or Aggregation Association, **Graph Walk** MUST emit the associated Class as an LHM row with `type=C`, PUSH that Class onto the LIFO list and active traversal path, and process its properties and descendants before continuing with another Association of the source Class.

When the current Class has no remaining eligible property, **Graph Walk** MUST POP it from the LIFO list and remove it from the active traversal path, then resume processing the preceding Class.

a) Processing of a root MUST end when its LIFO list and active traversal path are empty.

b) The root Class has level `1`. An emitted Attribute, Reference occurrence, or associated-Class occurrence has a level one greater than its containing Class; a copied `REF` Attribute has a level one greater than its containing `R` row.

7.3.4 Association semantics

Composition and Aggregation Associations MUST expand the associated concrete Class and its reachable properties. The incoming Association MUST supply the occurrence multiplicity.

A Reference Association MUST emit the reference occurrence and the target Class Attributes marked `identifier=PK`, copied as `REF`, without recursive traversal of the target Class. A missing primary key MUST be recorded as a non-fatal validation result.

The target Class MUST be resolved by Class semantic-model identifier; `association_role` distinguishes separate occurrences of the same target Class.

7.3.5 Cycles and repeated occurrences

A repeated Class semantic-model identifier in the active traversal path MUST terminate that branch and be reported as a cycle.

After a Class has been removed from the active traversal path, the same Class MAY be reached and traversed through another Association path or root. Such contextual reuse is permitted and does not constitute a cycle or a **BSM** design error.

Graph Walk MUST copy the originating **BSM** row id to source_bsm_id. The same source_bsm_id MAY therefore occur on multiple unique semantic_path values. A conformant **LHM** MUST satisfy the correspondence conditions in 5.3.6.

7.3.6 Semantic path

Every LHM row MUST have a deterministic `semantic_path` beginning with `$` and containing the complete path from its root Class. Every segment MUST be constructed in accordance with 5.6.2. For an associated-Class occurrence, the Association role and associated Class term are concatenated without an intervening separator and then normalised.

Graph Walk MUST reject a term that normalises to the empty string. Each `semantic_path` MUST be unique within the LHM and independent of execution environment or later syntax names. Distinct source terms that produce the same complete path, or differing content for one path, MUST be reported as `SEMANTIC_PATH_CONFLICT`.

7.3.7 Boundary between Graph Walk and post_graphwalk processing

Graph Walk is complete when every reachable occurrence for the supplied roots has been processed and the candidate 18-column LHM, including initial `local_name` and `xpath` values, has been produced.

LHM validation MUST apply the identity, canonical-path, reference, multiplicity, reproducibility, cycle, and content-conflict checks in 13.4. A failing candidate may be retained for diagnosis only. Initial duplicate (`module`, `local_name`) combinations MUST be reported but do not constitute semantic-

model failure. Preparation of the **LHM reviewed** may begin only after all semantic-model and structural failures have been corrected and the **candidate LHM** has been semantically approved.

7.3.8 LHM output contract

The LHM MUST use these 18 columns in this order: `sequence`, `module`, `level`, `type`, `identifier`, `name`, `datatype`, `multiplicity`, `association_role`, `definition`, `label_local`, `definition_local`, `source_bsm_id`, `semantic_path`, `associated_module`, `class_term`, `local_name`, and `xpath`.

The `type` column distinguishes Class, Attribute, Reference, and copied-key rows. `sequence` records output order; labels and definitions remain traceable to the **BSM**; `semantic_path` identifies the LHM row; and `source_bsm_id` identifies its source **BSM** row. **Graph Walk** derives the initial `local_name` mechanically from `name` and constructs the initial `xpath` from the emitted hierarchy by combining each row's `module` prefix with its initial `local_name`.

The candidate LHM produced by Graph Walk MUST remain unchanged. A separate LHM reviewed retains 18 columns but uses `value_domain` in column 9 in place of candidate-LHM `association_role`; it also permits only the controlled `local_name` adjustments and business-requirement-based `multiplicity` restrictions defined in 5.3.7. post_graphwalk MUST preserve `value_domain` and `semantic_path` without inference, repair, or regeneration, reject any non-canonical `semantic_path`, preserve the reviewed `local_name` and `multiplicity`, regenerate every `xpath` independently from the current hierarchy, `module`, and `local_name`, perform the remaining post-Graph Walk checks, and produce the 18-column HMD for taxonomy defined in 5.3.9. `domain_name` is not an LHM column in XBRL GL Next v0 (August 2026).

7.3.9 Graph Walk conformance

For the same conformant **BSM**, supplied root sequence, and **Graph Walk** rules, **Graph Walk** MUST reproduce exactly the same **candidate LHM** row order, content, `source_bsm_id`, and `semantic_path` values. A **candidate LHM** may be semantically approved only when the checks in 13.4 succeed.

7.4 Later root-specific syntax binding (HMD)

A root-specific **HMD** can be derived from an **HMD for taxonomy** when required by the selected syntax binding. The root is identified by its Class semantic-model identifier and uses the QName constructed from the applicable `module` and reviewed `local_name`.

HMD derivation preserves the **HMD for taxonomy** `source_bsm_id`, `semantic_path`, multiplicity, and row order.

Parts 2 and 3 define the serialisation-specific artefacts generated from an **HMD for taxonomy** under their fixed binding rules.

7.5 Implementation independence

Conformance is determined from the **FSM**, **BSM**, and **LHM** artefacts, the generated syntax-binding artefacts, and the observable transformations among them. Internal algorithms, program names, commands, programming languages, intermediate data, diagnostic formats, checksums, directory layouts, and ancillary output are implementation-specific.

Reference implementation authority: **Specialisation** is `tools/semantic/specialisation.py`, **Graph Walk** is `tools/semantic/graphwalk.py`, **Post-Graph Walk** is `tools/semantic/post_graphwalk.py`, and the sole Taxonomy Generator authority is `tools/taxonomy/xBRLGL_TaxonomyGenerator.py`. `tools/taxonomy/datatype_binding.py` is a required Generator dependency. `tools/semantic/validate_lhm.py` provides separate LHM validation. `tests/**` verifies the relevant processing programs.

7.6 Conformance test coverage (informative)

Useful test coverage includes inheritance, overrides, additions, deletion directives, Class-reference resolution, **Specialisation** and traversal cycles, multiple inheritance, Association validation, final-**BSM** identifiers, Reference Association projection, semantic-path correspondence, stable row order, multiple roots, and repeatable output.

8 Modules and the Palette Model

8.1 Module concept

A module is a governed semantic unit represented in the **FSM** and bound into one or more taxonomy modules. Module boundaries express semantic ownership and reuse scope.

A semantic module owns Class and property definitions; a taxonomy module owns their syntax declarations and relationships. Their correspondence is recorded through syntax binding.

8.2 Baseline module composition

Module	Primary role	Typical dependency
gl-cor	Core ledger structure and common anchors	Base module
gl-bus	Reusable party and general business semantics	gl-cor
gl-muc	Multicurrency amounts and exchange information	gl-cor
gl-taf	Tax structures and reporting semantics	gl-cor; jurisdiction tax modules
gl-btx	Orders, shipments, invoices, payments, and source events	gl-cor and gl-bus where applicable
gl-ehm	Enhanced measurable items and traceable quantities	gl-cor and gl-bus where applicable
gl-lnk	Links to evidence and end-reporting structures	gl-cor
gl-sta	Statistical and other non-financial reporting	gl-cor and gl-ehm where applicable

The table states architectural roles. The **FSM** declares module and root semantics; root-specific **HMDs** and the fixed binding rules determine the taxonomy modules and message entry points generated.

8.3 Module dependencies and ownership

Each module records its dependencies and ownership. Reused Classes and properties retain the module of their semantic owner.

8.4 Taxonomy assembly realisation

A concrete message, ledger, or report type has an applicable root, root-specific binding, and message entry point. Examples include Journal Entry, Trial Balance, Order, Shipment, Invoice, Receipt, and Payment.

The generated taxonomy artefacts identify the relationship from the root Class semantic-model identifier through the syntax-binding root QName to the message entry point.

One taxonomy package MAY expose several message entry points when it supports several root-specific structures.

Taxonomy generation is reproducible from the formal **HMDs** and the fixed syntax-binding rules. The generated package identifies its message entry points.

8.5 Palette model

The Palette model assembles reusable taxonomy modules into the syntax binding and message entry points determined by the **HMD for taxonomy** and the fixed binding rules.

A conformant palette assembly preserves module namespace ownership and semantic traceability to the **HMD for taxonomy** and its source LHM artefacts.

9 Palette Taxonomy

9.1 Purpose

A palette taxonomy is a modular XBRL implementation generated from an **HMD for taxonomy** under the applicable fixed syntax-binding rules.

The Palette model is the assembly approach; a taxonomy module is a reusable syntax component; and a message entry point is the generated physical taxonomy artefact for a selected root.

9.2 Tuple palette taxonomy

The XBRL 2.1 entry point imports the required concept and content-model schemas and loads the associated linkbases. Tuple containment, XML Schema occurrence constraints, and linkbase relationships are derived from the same reviewed LHM rows and multiplicities used by other bindings.

For each module namespace, the module's element-declaration schema and content-model schema form one schema document set. An element-declaration schema such as `gl-cor.xsd` is included by the content-model schema of the same target namespace, such as `gl-cor-content.xsd`. A palette-specific content-model schema references a shared module having a different target namespace by import.

Where the same underlying **BSM** structure is instantiated as separate palette-specific declarations, the **conformant taxonomy generator** MUST generate the applicable element declarations and type definitions separately in each palette-specific namespace. A deliberately shared declaration remains in its shared module.

9.3 OIM-oriented taxonomy

The OIM-oriented entry point implements **HMD** occurrences using primary items, hypercubes, dimensions, domains, members, roles, and relationships as appropriate. xBRL-CSV metadata binds CSV tables and columns to that taxonomy; metadata is a report binding, not a semantic-model layer.

9.4 Message and DTS entry points

A message entry point loads the module versions, extension relationships, code-list taxonomies, labels, references, and validation rules required by its root-specific structure.

The message entry point identifies the root QName and binds the hierarchy defined by the **HMD for taxonomy**. One taxonomy package MAY contain several message entry points.

A package can expose an aggregate discovery entry point while retaining root-specific validation through its message entry points.

10 How to create extensions within the XBRL GL Next Taxonomy Framework

10.1 Define the semantic extension

Define or revise the extension in an **FSM** module under the maintainer's namespace. State whether it is Shared, Aligned, or Distinct; identify its base Class or property where applicable; and define terms, datatype, associated Class, multiplicity, definition, labels, code-list dependency, source, and rationale.

10.2 Validate and specialise

Validate Class references, inheritance, deletions, Abstract Class usage, code-list references, and final-**BSM** duplicate/conflict combinations. Run **Specialisation** to produce a **BSM** and confirm that no instruction rows or unresolved references remain.

10.3 Generate hierarchies

Generate the **candidate LHM** from each intended root and validate stable ordering, `semantic_path` uniqueness, `source_bsm_id` resolution and permitted `source_bsm_id` reuse, path construction, Reference Association results, multiplicity, cycle termination, reproducibility, and absence of content conflicts. Correct any **FSM** or **BSM** design error and rerun **Graph Walk** before semantic-model approval. **Graph Walk** reports initial duplicate (`module`, `local_name`) combinations; those duplicates are resolved by permitted `local_name` adjustments in the **LHM reviewed**.

10.4 Generate taxonomy bindings

Use the **HMD for taxonomy** and the fixed rules in Parts 2 and 3 to generate the XBRL 2.1 or OIM syntax-binding artefacts. Generated artefacts preserve semantic traceability to the **FSM**, **BSM**, and **LHM**.

11 Syntax Binding

11.1 Binding source

Syntax binding starts from one or more formal **HMD_for_taxonomy** files and follows the fixed rules of the applicable binding Part.

The **FSM** remains the semantic source; the **BSM** and **LHM** are derived semantic artefacts; and **HMD**, taxonomy schemas, linkbases, JSON metadata, and CSV layouts are syntax-binding artefacts generated under fixed Framework rules.

11.2 Binding information

For each serialised LHM row, the binding records `source_bsm_id`, `semantic_path`, the reviewed concept or dimension QName, syntax path or CSV expression, multiplicity and datatype mappings, selector conditions, and code-list implementation.

11.3 XBRL 2.1 and OIM equivalence

XBRL 2.1 tuple nesting and OIM dimensional structures can use different physical constructs. Each binding identifies the LHM rows and **semantic paths** it implements, and each XBRL item QName corresponds to one reviewed semantic-model concept.

11.4 Validation

Binding validation includes DTS discovery, schema and linkbase validation, representative instance validation, semantic-path-to-concept completeness, code-list conformance, and comparison of fact values, units, periods, entities, dimensions, and reference relationships across supported serialisations.

12 Reverse engineering and migration support (informative)

12.1 Purpose

This chapter defines the conceptual foundation by which the XBRL GL Next framework supports reverse engineering and migration from tuple-based XBRL GL (XBRL 2.1) to OIM-based serialisations (for example, xBRL-CSV), while preserving semantic integrity and traceability.

This chapter establishes principles and modelling assumptions.

Detailed procedural and conformance requirements for migration are specified in Part 4 of this framework.

12.2 Background: XBRL GL XML (XBRL 2.1 tuple-based realization) and implicit semantics

Tuple-based XBRL GL is a syntax-oriented design that utilises XML Schema `complexType` definitions and tuple nesting to represent accounting and business data structures.

In this design:

- hierarchical relationships are expressed through tuple containment;
- multiplicities are expressed through XML Schema cardinalities;
- structural extension is achieved by extending `complexType` definitions.

Although primarily syntax-driven, XBRL GL XML (XBRL 2.1 tuple-based realization) implicitly embodies a logical hierarchical model (LHM) corresponding to the `complexType` structure defined in the XML schema. Header–detail groupings, document-level constructs, and repeating line structures can therefore be interpreted as semantic structures, even though they are not explicitly defined as a semantic model.

12.3 Principle of semantic primacy

The xBRL GL framework is based on the principle that:

Semantics are authoritative; serialisations are bindings.

Accordingly:

- OIM-based xBRL-CSV are treated as equivalent serialisations of the same semantic model;
- migration between serialisations is achieved by recovering and reusing semantic layers, rather than by direct syntax-to-syntax transformation.

12.4 Role of the Foundational Semantic Model (FSM)

To externalise and stabilise the semantics embedded in tuple-based structures, the framework defines a Foundational Semantic Model (**FSM**).

The FSM:

- defines reusable abstract classes (for example, Header and Detail);

- defines common properties independently of XML Schema constructs;
- provides a stable semantic vocabulary shared across domains and modules.

These abstract classes enable structural extension through specialisation, allowing concrete classes (such as `entryHeader`, `transactionHeader`, or `reportHeader`) to inherit common semantics while introducing domain-specific refinements.

12.5 Reconstruction of semantic layers

Reverse engineering is conceptually supported by reconstructing semantic layers from existing tuple-based artefacts as follows:

a) Logical Hierarchical Model (**LHM**)

Tuple containment and cardinality are analysed to reconstruct hierarchical Header–Detail structures and multiplicity constraints equivalent to those produced by the forward graph-walk process.

b) Business Semantic Model (**BSM**)

Reconstructed hierarchies are flattened into class and property definitions, and specialisation relationships are identified.

c) Alignment with the **FSM**

Recovered Classes and properties are aligned with the **FSM** and Aligned Pool, ensuring reuse and preventing duplication.

This reconstruction restores the semantic model as an explicit, syntax-independent artefact.

12.6 Regeneration and dual syntax support

Once semantic layers have been reconstructed and aligned:

- XBRL GL XML (XBRL 2.1 tuple-based realization) taxonomies may be regenerated to ensure consistency;
- OIM-based taxonomies (for example, xBRL-CSV) may be generated from the same semantic source.

This enables coexistence and parallel operation of legacy and modern serialisations during transition periods.

12.7 Summary

Reverse engineering in the xBRL GL framework is not a conversion between syntaxes.

It is a semantic recovery and regeneration process that enables existing implementations to evolve toward OIM-based representations while preserving meaning, extensibility, and auditability.

13 Conformance

13.1 General

Conformance is assessed from the observable artefacts and relationships defined in this Part. An implementation does not need to use a prescribed programming language, internal data structure, command interface, or algorithm, but its outputs MUST satisfy the applicable contracts and deterministic processing results.

13.2 FSM conformance

A conformant **FSM** MUST use the 15-column contract in 5.3.2 and 7.2, use valid semantic-model identifiers, contain resolvable Class references, and express extension through applicable Shared,

Aligned, and Distinct definitions. Association rows MUST satisfy the Association content rules. The **FSM** contract contains only semantic-model information.

13.3 BSM conformance

A conformant **BSM** MUST be deterministically generated by resolving only **Specialisation** Associations from conformant **FSM** inputs. It MUST use the 16-column contract, assign a unique stable `id` to every effective row, contain no unresolved **Specialisation** Association or multiplicity-zero directive, retain effective Composition, Aggregation, and Reference Associations unresolved for **Graph Walk**, and satisfy the duplicate, conflict, Class-reference, and concrete-Class rules in 7.2.

13.4 LHM conformance

Graph Walk output is a **candidate LHM**. A **candidate LHM** may be semantically approved only when all of the following semantic-model and structural conditions are satisfied:

- every row records the correct originating **BSM** id as source_bsm_id and a complete canonical semantic_path satisfying 5.6.2;
- each semantic_path is unique and corresponds to exactly one source_bsm_id;
- every source_bsm_id resolves to an existing **BSM** row;
- a source_bsm_id MAY occur in multiple rows when those rows have different semantic_path values;
- reuse of a `source_bsm_id` on different unique `semantic_path` values is not a conformance error;
  the candidate uses the 18-column contract defined in 5.3.5 and 7.3.8, with the prescribed rows and order;
  its hierarchy, occurrence order, row types, identifiers, references, datatypes, multiplicities, definitions, and Association context satisfy the applicable **Graph Walk** rules;
  every initial `local_name` is a valid XML Schema NCName, and every initial `xpath` is correctly constructed from the emitted hierarchy by combining the prefix registered for each row's `module` with its initial `local_name`; and
  the candidate is reproducible and has not been manually modified. Initial duplicate (`module`, `local_name`) combinations MUST be reported but do not prevent semantic-model approval.

A candidate MAY be retained for diagnosis. Semantic approval and use as the source of an **LHM reviewed**, **HMD for taxonomy**, or conformant syntax binding require every applicable condition to pass.

13.5 Execution-record and post_graphwalk conformance

A conformant **Post-Graph Walk** result MUST contain one formal HMD_for_taxonomy for each effective `level = 1` root. **Post-Graph Walk** processing MUST validate each `semantic_path` against 5.6.2, reject a non-canonical value without repairing or regenerating it, and preserve each accepted `semantic_path` unchanged. It MUST preserve every effective row and every value other than regenerated `xpath`, preserve the reviewed `local_name` and `multiplicity`, regenerate `xpath` separately within each HMD from the current hierarchy, `module`, and `local_name`, enforce (`module`, `local_name`) uniqueness within each HMD, and verify the definition consistency of each QName reused across HMDs. It MUST satisfy the checks in 7.1. `domain_name` is absent from every LHM table for XBRL GL Next v0 (August 2026).

13.6 Processing conformance

A conformant **Specialisation** process MUST satisfy 7.2. A conformant **Graph Walk** process MUST satisfy 7.3. Equivalent internal implementations are conformant when they produce the same required observable results.

Appendix A (informative) Example: Extending “Sales Order” and “Purchase Order” from abstract class “Order”

A.1 Two-phase architecture

In audit data services, the focus is not on the business transaction document itself—such as an order—but on capturing the context of the transaction within an organization. This includes information about which department processed the transaction, how it was recorded in the accounting ledger, who handled registration and approval, and the accounting period and account codes used for journal entries. The same abstract class order requires different contextual information depending on whether it is viewed from the customer’s or supplier’s perspective.

The logical model for audit data services defines common business documents as abstract classes in the **FSM**. These abstract definitions are then specialised into subclasses in the **BSM** representing how the document is registered in each business system, such as Purchase Order in the customer’s system or Sales Order in the supplier’s system. This structured approach enhances versatility and extensibility within the model.

Extended subclass definitions include information relevant to audit data, such as common elements like Activities and Accounting Period, as well as business-specific details like Purchaser in the customer's system or Sales Person in the supplier's system. This structured representation ensures consistency while allowing system-specific adaptations.

The two-phase architecture ensures clear traversal by processing only the association end leading to the associated class, eliminating ambiguities from bidirectional relationships. This approach introduces deterministic specialisation and inheritance processing, unlike original HL7, which lacks clear rules for specialisation, multiplicity constraints, and association directionality.

A.1.1 Specialisation Processing

Specialisation is resolved first to establish inheritance before processing attributes and associations. This involves:

— Inheritance Resolution:

Sales Order: Inherits attributes from Order but removes Supplier (0) and adds Sales Person.

Purchase Order: Inherits attributes from Order but removes Customer (0) and adds Purchase Clerk.

All properties from the superclass Order (Order Number, Order Date, Transaction Amount, Seller, Buyer) are copied into **BSM**.

— Apply **Specialisation** Rules:

Sales Order: Removes Supplier, adds Sales Organization and Sales Person.

Purchase Order: Removes Customer, adds Purchase Organization and Purchase Clerk.

— Finalize Class Structure:

Complete structure definition before proceeding to attribute and association traversal.

This is the first phase, in which **Specialisation** generates the specialised subclasses in the **BSM**.

- In this process, the abstract business transaction Classes defined in the **FSM** are resolved into their effective **BSM** Classes by **Specialisation**.
- Based on extension conditions:

- — Some properties may be removed.
- — Some properties can be modified, including changes in multiplicity, datatype, or Association type. An Association role change is represented by deleting the Association identified by the former role and defining a new Association identified by the new role.
- — New properties can be added. These directives are resolved by **Specialisation** before **Graph Walk** begins.

This ensures that **BSM** subclasses inherit and refine properties from **FSM** while preserving both flexibility and structural integrity.

A.1.2 Graph Walk Processing

Once specialisation is complete, the **Graph Walk** proceeds by processing attributes and relationships based on the finalized structure:

The **Graph Walk** ensures specialisation is handled first, resolving inheritance before processing attributes and associations. This approach allows for the correct removal and addition of properties in the Sales Order and Purchase Order audit data models.

Table A.1 — FSM/BSM two-layer architecture

Customer Specialised subclass in BSM	Business Transaction (Abstract class in FSM)	Supplier Specialised subclass in BSM
Purchase Order	Order	Sales Order
Purchase Materials Received (Goods Receipt Ledger)	Delivery Note	Sales Shipment Made (Delivery Ledger)
Purchase Invoice	Invoice	Sales Invoice
Accounts Payable Payment Made	Bank Transfer Note	Accounts Receivable Cash Received

A.2 Abstract class 'Order' and its specialisation

Table A.2 lists the abstract class Order and its subclasses Purchase Order and Sales Order in **FSM**. The properties of Order are categorized into transaction document-oriented data, application-oriented data, and audit-purpose data.

- Transaction document-oriented data include:
  - — Unique identifiers such as ID, Date, Payment Term, and Settlement Term.
  - — Trading partners, Customer and Supplier.
- Audit-purpose data include:
  - — Accounting Period: Specifies the relevant period for the transaction.
  - — Activities: Captures processes related to the transaction.
  - — Business Segment: Identifies specific operational segments, such as departments or product lines.
- Application-oriented data are subclass-specific:
  - — Purchase Order includes supplier-specific data such as Purchase Department and Purchaser.
  - — Sales Order includes sales-specific data such as Sales Organization and Sales Person.
- Additionally, trivial data are removed in subclass specialisation:
  - — Supplier is removed from Sales Order.
  - — Customer is removed from Purchase Order.

Table A.2 — Order, Purchase Order, and Sales Order before specialisation in FSM

Property Type	Property term	Association role	Associated class	Multiplicity		
				Super class (Order)	Purchase Order	Sales Order
Attribute	ID (PK)			1	inherit	inherit
Attribute	Date			0..1		
Reference Association		Reference	Customer	0..1		
Reference Association		Reference	Supplier	0..1		
Composition		Transaction	Amount	0..1		
Reference Association			Payment Term	0..1		
Reference Association			Settlement Method	0..1		
Composition			Activities	undefined	0..1	0..1
Reference Association			Accounting Period		0..1	0..1
Reference Association			Business Segment		0..*	0..*
Specialisation			Order	undefined	1	1
Reference Association		Reference	Customer		0[a)]	undefined
Reference Association		Reference	Supplier		undefined	0 [b)]
Reference Association		Purchase Organization	Business Segment		0..1	undefined
Reference Association		Sales Organization	Business Segment		undefined	0..1
Reference Association		Purchaser	Employee		0..1	undefined
Reference Association		Salesperson	Employee		undefined	0..1

a) Customer is removed from Purchase Order.
b) Supplier is removed from Sales Order.

Specialisation copies all properties of the superclass to the subclass except those with a specified multiplicity of 0, indicating removal. There is no abstract class in **BSM**. If a member body defines an aligned extension by further specialising the specialised subclasses Purchase Order and Sales Order, specialisation allows the removal, modification, and addition of properties. The resulting **BSM** property table is listed in **Table A.3**.

Table A.3 — Order, Purchase Order, and Sales Order after specialisation in BSM

Property Type	Property term	Association role	Associated class	Multiplicity	
				Purchase Order	Sales Order
Attribute	ID (PK)			1	1
Attribute	Date			0..1	0..1
Reference Association		Reference	Customer	— (deleted)	0..1
Reference Association		Reference	Supplier	0..1	— (deleted)
Composition		Transaction	Amount	0..1	0..1
Reference Association			Payment Term	0..1	0..1
Reference Association			Settlement Method	0..1	0..1
Composition			Activities	0..1	0..1
Reference Association			Accounting Period	0..1	0..1
Reference Association			Business Segment	0..*	0..*
Specialisation (resolved; no effective BSM row)			**Order**	**resolved**	**resolved**
Reference Association		Purchase Organization	Business Segment	0..1	— (not defined)
Reference Association		Sales Organization	Business Segment	— (not defined)	0..1
Reference Association		Purchaser	Employee	0..1	— (not defined)
Reference Association		Salesperson	Employee	— (not defined)	0..1

Table A.3 is a comparison of **Specialisation** outcomes and is not a serialisation of the effective **BSM** row set. A cell marked "— (deleted)" or "— (not defined)" records the comparison outcome only. The row describing the **Specialisation** Association is retained only to show that it was resolved; **Specialisation** Associations, deletion directives, and non-applicable properties are absent from the actual **BSM** output.

Appendix B
(informative)
Example of the normal navigation Graph Walk

B.1 Graph Walk: Defining the LHM by copying the BSM

Figure B.1 is an example object class diagram. Associations include reference associations, aggregations, and compositions.

A reference association has a multiplicity of "0..1," and all listed attributes are optional.

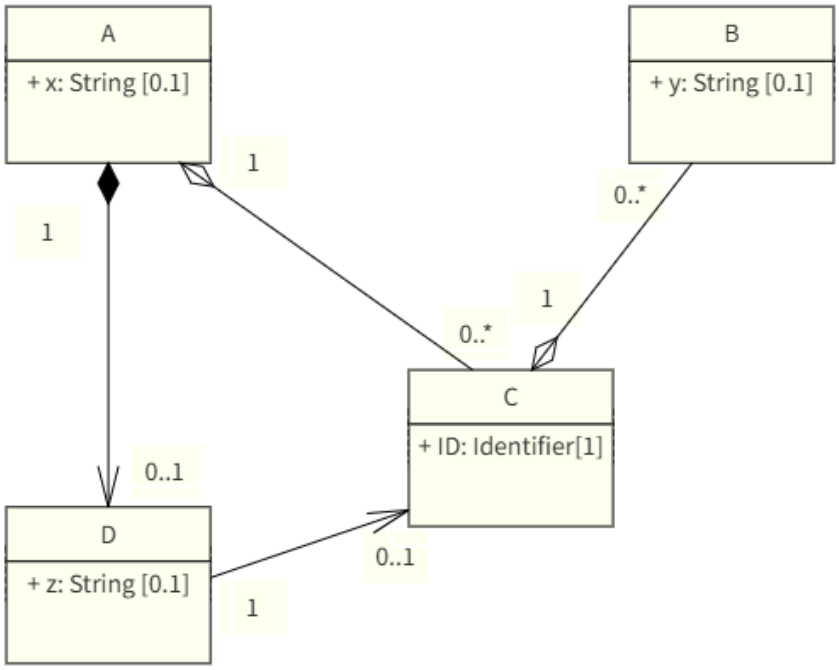


Figure B.1 — Example object class in BSM

Figure B.1 represented the object class diagram of this example.

Table B.1 — Properties of object classes in BSM

No.	Class	Property Type	Property term	Association role	Representation term	Associated class	Multiplicity
1	A	Attribute	x		String		0..1
2	A	Aggregation				C	0..*
3	A	Composition				D	0..1
4	B	Attribute	y		String		0..1
5	C	Attribute	ID		Identifier		1
6	C	Aggregation				B	0..*
7	D	Attribute	z		String		0..1
8	D	Reference Association				C	0..1

NOTE This is a conceptual example. Module identifiers are not defined, and the intermediate LHM tables show only seven illustrative columns rather than the normative 18-column LHM contract. Consequently, `source_bsm_id`, `semantic_path`, `local_name`, and `xpath` are not shown and cannot be validated from this example.

The root class is A, and the **Graph Walk** starts from there.

The **Graph Walk** navigates using the following steps:

1) Step 1: Copying Attributes

Each class's attributes are copied completely before traversing to associated classes.

When a class is visited, all attributes are copied first before navigating any associations.

2) Step 2: Navigating Associations

After copying Attributes, Composition and Aggregation Associations are processed by traversing one level deeper. Reference Associations are processed without traversing the associated Class.

For a Composition or Aggregation Association, the associated Class is placed onto the LIFO list and copied to the target model, traversing one level deeper, and steps 1 and 2 are recursively performed.

a) Cycle Prevention

If the associated class is already in the LIFO list, stop navigation to prevent cyclic traversal.

b) Reference Associations

Copy only the unique identifier of the associated Class, which becomes the reference identifier in the LHM.

The associated class is not traversed further.

c) Composition or Aggregation Associations

Add the associated class to the top of the LIFO list and copy all its properties.

Traverse the associated class recursively.

Once all properties of the class are copied and consumed, remove it from the LIFO list.

B.2 The Graph Walk process for the root class A

- **Navigate A/ LIFO list: [A]/ Current level 2**

— Step 1: Copy all attributes of class A.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1

— Step 2: Check the associated classes C and D. Based on the selection order defined by the multiplicity rule, select D first, then navigate to C. Place D on top of the LIFO list and proceed with the **Graph Walk** to D.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1

- **Navigate D/ LIFO list: [A, D]/ Current level 3**

— Step 1: Copy all attributes of class D.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1

— Step 2: Identify the Reference Association from D to C. Emit the Reference occurrence without placing C on the LIFO list.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1

- **Process the D-to-C Reference Association / LIFO list: [A, D] / Current level 3**

— Step 1: Copy the unique identifier of Class C and complete processing of the Reference Association from D to C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1
6	4	A (Attribute)	REF	>>> ID	Identifier	1

NOTE 1 In this case, only the unique identifier is copied as a reference identifier.

— Step 2: The Reference Association is complete. C was not added to the LIFO list; continue **Graph Walk** with Class D.

- **Continue navigating D / LIFO list: [A, D] / Current level 3**

There is no other property and finish navigation on D. Remove class D from the LIFO list and proceed with the **Graph Walk** to class A.

- **Continue navigating A/ LIFO list: [A]/ Current level 2**

— Step 1: No attributes remain to be copied

— Step 2: After navigating D, the remaining class is C. Check class C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1
6	4	A (Attribute)	REF	>>> ID	Identifier	1
7	2	C (Aggregation)		> C		0..*

- **Navigate C/ LIFO list: [A, C]/ Current level 3**

— Step 1: Copy all attributes of class C.

NOTE 2 If an association from A to C is an aggregation, copy all properties of C.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> X	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1
6	4	A (Attribute)	REF	>>> ID	Identifier	1
7	2	C (Aggregation)		> C		0..*
8	3	A (Attribute)	PK	>> ID	Identifier	1

— Step 2: Check the associated classes B. Place B on top of the LIFO list and proceed with the **Graph Walk** to B.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		> x	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1
6	4	A (Attribute)	REF	>>> ID	Identifier	1
7	2	C (Aggregation)		> C		0..*
8	3	A (Attribute)	PK	>> ID	Identifier	1
9	3	C (Composition)		>> B		0..*

- **Navigate B/ LIFO list: [A, C, B]/ Current level 4**

— Step 1: Copy all attributes of class B.

No.	Level	Type	Identifier	Name	Data Type	Multiplicity
1	1	C (Class)		A		
2	2	A (Attribute)		X	String	0..1
3	2	C (Composition)		> D		0..1
4	3	A (Attribute)		>> z	String	0..1
5	3	R (Reference Association)		>> C		0..1
6	4	A (Attribute)		>>> ID	Identifier	1
7	2	C (Aggregation)		> C		0..*
8	3	A (Attribute)		>> ID	Identifier	1
9	3	C (Composition)		>> B		0..*
10	4	A (Attribute)		>>> y	String	0..1

— Step 2: Traversal of Class B is complete. Remove Class B from the LIFO list and resume **Graph Walk** with Class C.

- **Continue navigating C/ LIFO list: [A, C]/ Current level 3**

There is no other property and finish navigation on C. Remove class C from the LIFO list and proceed with the **Graph Walk** to class A.

- **Continue navigating A/ LIFO list: [A]/ Current level 2**

There are no remaining properties, completing the navigation for class A. Remove class A from the LIFO list and finalize the **Graph Walk** for class A.

The **Graph Walk** traverses an association in a deterministic depth-first search manner to reach the associated class without applying any conditional rules

Appendix C
(informative)
Semantic Layers Example: defining an extension in the FSM and generating the resulting BSM and LHM

C.1 Purpose and scope

This annex shows how an extension is introduced through applicable **FSM** inputs, resolved by **Specialisation** into the **BSM**, and transformed by **Graph Walk** into a **candidate LHM**. Preparation of the **LHM reviewed**, post_graphwalk processing, **HMD** preparation, and syntax binding occur only after the **candidate LHM** has been semantically approved.

The example is based on the layered semantic architecture used by the XBRL GL Next Taxonomy Framework to keep business data consistent, comparable, and traceable across supported serialisations.

Supported serialisations in this framework are:

- XBRL 2.1 / XML (Part 2); and
- OIM / xBRL-CSV (Part 3).

C.2 Inputs (example sheets)

The example uses the following semantic-model layer files (CSV exports from the **FSM**, **BSM**, and **LHM** sheets):

- Base **FSM**: `FSM.csv` (jurisdiction-neutral abstract classes and properties; e.g., "Communication").
- **FSM** extension (Business Transactions): `FSM_btx.csv` (introduces `Business Transactions` semantics linking operational documents to the ledger).
- **BSM**: `BSM.csv` (specialised refinement of selected **FSM** concepts; maintains traceability to **FSM**).
- **LHM** (btx): `LHM.csv` (hierarchical structure generated from the **BSM**; includes hierarchy levels, row types, the originating **BSM** `id` as `source_bsm_id`, and `semantic_path`).

C.3 Design policy used in the example

a) The **FSM** provides the authoritative semantic base and may contain Shared, Aligned, and Distinct definitions. Shared definitions provide reusable concepts such as Entity, Party, Person, Account, Amount, and Communication.
b) Aligned extensions are introduced as **FSM** inputs when domain-specific or jurisdiction-specific semantics are required beyond the Shared definitions. Where cross-jurisdictional reuse is intended, the Aligned Pool preserves semantic equivalence and comparability.
c) **Specialisation** resolves only **Specialisation** Associations and produces the effective **BSM** by applying inheritance, additions, overrides, multiplicity constraints, and deletion directives. All other Association types remain unresolved in the **BSM** for **Graph Walk**.
d) **Graph Walk** deterministically processes the complete **BSM** from one or more supplied root Classes. Composition and Aggregation Associations expand the candidate hierarchy; Reference Associations emit reference rows without recursively processing the referenced Class.
e) **Semantic Path** is the common semantic anchor. Every **LHM** row records its complete occurrence path for comparison with later binding artefacts. **Semantic Path** does not define an XML XPath or XBRL QName.

C.4 Step-by-step procedure (FSM → BSM → LHM)

C.4.1 Step 1 — Define the extension in the FSM

Goal: Introduce Business Transactions semantics by specialising existing Shared Classes. The Shared **FSM** definitions remain unchanged; additions, overrides, multiplicity constraints, and deletion directives declared for each child Class are resolved in the resulting **BSM**.

Procedure (example pattern):

a) Create an **FSM** extension file for the module, here `btx`, containing the Classes, Attributes, and Associations required for the Business Transactions domain, including an appropriate root Class.
b) Ensure that the extension is consistent with the modular architecture. In this example, the Business Transactions module has its own root Class and is not anchored to the Accounting Entries root Class.
c) Where cross-jurisdictional reuse is intended, manage the extension as an Aligned extension through the applicable Aligned Pool governance mechanism to maintain semantic equivalence and avoid duplication.

C.4.2 Step 2 — Apply Specialisation to create the BSM

Goal: Produce the effective **BSM** by resolving the **Specialisation** Associations defined in the base **FSM** and applicable **FSM extensions**.

Procedure (example pattern):

a) Apply **Specialisation** to the complete set of applicable **FSM** inputs. **Specialisation** resolves only **Specialisation** Associations and deterministically produces the effective **BSM**. The generated **BSM** is not independently selected or modified.
b) Apply the declared inheritance, additions, overrides, multiplicity constraints, and deletion directives. A Property may be removed from the effective **BSM** by an applicable multiplicity-zero directive.
c) Example: the general Party Class may be specialised as the `btx_EntityParty` Class. An inherited Attribute such as `cor_PartyID` receives its effective **BSM** `id` while retaining its semantic relationship with the originating **FSM** definition.
d) Align this step with the “avoid semantic misuse” principle: core concepts should be defined and extended so implementers do not place person attributes into verification-metadata or “closest available” fields.

(For the driver requirement in the requirements document, see “first-class party/person identification”.)

C.4.3 Step 3 — Generate the LHM by deterministic Graph Walk

Goal: Produce the complete **candidate LHM** with stable **Semantic Paths** before **HMD** preparation, binding, or taxonomy generation.

Procedure (example pattern):

a) Perform deterministic depth-first **Graph Walk** from the supplied root Classes. Process Composition, Aggregation, and Reference Associations according to their respective **Graph Walk** rules.
b) Assign a `semantic_path` to every emitted LHM row in accordance with 5.6.2. The path begins with `$`, contains the root, preserves the exact sequence of semantic-model identifiers corresponding to the Associations traversed by **Graph Walk**, and uses only the LHM module value, one module-term underscore, and ASCII letters in each normalised term.
c) After **Graph Walk** has produced the **candidate LHM** with initial `local_name` and `xpath` values, perform the prescribed semantic-model and structural LHM conformance checks. Controlled `local_name` adjustments and business-requirement-based `multiplicity` restrictions

belong to the separate **LHM reviewed**. post_graphwalk validates and preserves `semantic_path` without repair or regeneration, preserves the current `local_name` values, and regenerates `xpath` for every row independently from `semantic_path`. `domain_name` is not used in the v0 LHM contracts.

d) Record hierarchy level, row type, Association role where applicable, originating **BSM** `id` as `source_bsm_id`, and `semantic_path` in the **candidate LHM**. Generate the initial `local_name` from `name`, then generate the initial `xpath` from the emitted hierarchy by combining each row's `module` prefix with its initial `local_name`. Do not add an independent LHM `id` or `domain_name`.

Worked example (from the annexed description in Part 1):

- **Semantic Path** example: `$.btx_BusinessTransactions.btx_EntityParty.cor_PartyID`
- XML binding example: the initial XPath is recorded by **Graph Walk** and is regenerated by post_graphwalk from the current reviewed `local_name` values before taxonomy generation.

C.5 Conformance checks recommended for this workflow

When producing/maintaining these layers, the following checks are expected as baseline:

- **LHM** identity check: each `semantic_path` MUST correspond to exactly one `source_bsm_id`. A `source_bsm_id` MAY occur on multiple rows when the originating **BSM** row is reached through different unique **Graph Walk** paths. A duplicate `semantic_path`, unresolved `source_bsm_id`, or conflicting row content is an **FSM** or **BSM** design error.
- **Semantic Path** conformance: each path MUST include the root, preserve the exact sequence of semantic-model identifiers corresponding to the Associations traversed by **Graph Walk**, and satisfy the canonical segment and normalisation rules in 5.6.2. Empty normalised terms and normalisation collisions are errors.
  Reproducibility: the same conformant **BSM**, supplied root sequence, and **Graph Walk** rules MUST produce the same **candidate LHM**.
  Post-Graph-Walk separation: permitted `local_name` adjustments and business-requirement-based `multiplicity` restrictions MUST be recorded in a separate **LHM reviewed**.
  post_graphwalk MUST validate and preserve `semantic_path` unchanged and MUST regenerate `xpath` independently from the current hierarchy, `module`, and `local_name` for the **HMD for taxonomy**. The **candidate LHM** MUST remain unchanged.

C.6 Expected outputs and how they are used

- **FSM** and applicable **FSM extensions**: the authoritative semantic-model inputs comprising applicable Shared, Aligned, and Distinct definitions.
- **BSM**: the effective semantic model produced after inheritance, additions, overrides, multiplicity constraints, and deletion directives declared through **Specialisation** Associations have been resolved.
- LHM: the complete candidate hierarchy generated deterministically from the **BSM**. A candidate containing a non-canonical or duplicate `semantic_path`, an empty normalised term, a normalisation collision, an unresolved `source_bsm_id`, or conflicting row content may be retained for diagnosis only. It may be semantically approved only after the source or processing defect has been corrected and **Graph Walk** has been rerun successfully. Reuse of one `source_bsm_id` on different unique `semantic_path` values is permitted.
- Semantic-model artefact set: the **FSM** source is the authoritative input; the **BSM** result and **candidate LHM** are deterministic generated artefacts. The **LHM reviewed** records the responsible model definer's permitted review adjustments, while subsequent processing follows fixed Framework rules.

Appendix D (informative) Example: “VAT category taxable amount” (EN 16931 BT-116)

D.1 UBL 2.1 (Peppol BIS Billing 3.0 – UBL Invoice)

In UBL, VAT breakdown is represented by one or more `cac:TaxSubtotal` elements under `cac:TaxTotal`. The VAT category taxable amount is the `cbc:TaxableAmount` within each `cac:TaxSubtotal`, and its meaning is explicitly “sum of all taxable amounts subject to a specific VAT category code and rate”.

Typical XPath (illustrative):

```
/Invoice
  /cac:TaxTotal
  /cac:TaxSubtotal[
      cac:TaxCategory/cac:TaxScheme/cbc:ID = 'VAT'
      and cac:TaxCategory/cbc:ID           = 'S'
      and cac:TaxCategory/cbc:Percent      = 10
    ]
  /cbc:TaxableAmount
```

Notes:

- The selector uses `cac:TaxScheme/cbc:ID='VAT'`, plus `cac:TaxCategory/cbc:ID` (category code) and, where present, `cbc:Percent` (rate).
- `cbc:TaxableAmount/@currencyID` aligns with the document-currency validation rules of the applicable binding (for example, invoice currency).

D.2 UN/CEFACT CII D16B (EN 16931 CII)

In CII, the VAT breakdown is carried in repeated `ram:ApplicableTradeTax` elements under header settlement. The VAT category taxable amount (BT-116) is carried in `ram:BasisAmount` within each `ram:ApplicableTradeTax` (with the category and rate carried alongside it). This is shown explicitly in CII guidance where `ram:BasisAmount` is labelled as “BT-116”.

Typical XPath (illustrative):

```
/rsm:CrossIndustryInvoice
  /rsm:SupplyChainTradeTransaction
  /ram:ApplicableHeaderTradeSettlement
  /ram:ApplicableTradeTax[
      ram:TypeCode                 = 'VAT'
      and ram:CategoryCode         = 'S'
      and ram:RateApplicablePercent = 10
    ]
  /ram:BasisAmount
```

Notes:

- `ram:TypeCode='VAT'` identifies VAT when several tax types can occur.
- Category and rate selection uses `ram:CategoryCode` and `ram:RateApplicablePercent`.

D.3 Example syntax binding table entry

Column	Value
Root	Invoice

Column	Value
Semantic group	VAT BREAKDOWN(BG-23)
Semantic term	VAT category taxable amount(BT-116)
Semantic path	$.inv_Invoice.inv_VATBreakdown.inv_VATCategoryTaxableAmount

UBL 2.1 target (XPath)	`/Invoice`
	`/cac:TaxTotal`
	`/cac:TaxSubtotal[`
	`cac:TaxCategory/cac:TaxScheme/cbc:ID = 'VAT'`
	`and cac:TaxCategory/cbc:ID = $vatCategoryCode`
	`and cac:TaxCategory/cbc:Percent = $vatRate`
	`]`
	`/cbc:TaxableAmount`

UBL value / attribute	`` `cbc:TaxableAmount` ; currency: `@currencyID` ``

CII target (XPath)	`/rsm:CrossIndustryInvoice`
	`/rsm:SupplyChainTradeTransaction`
	`/ram:ApplicableHeaderTradeSettlement`
	`/ram:ApplicableTradeTax [`
	`ram:TypeCode = 'VAT'`
	`and ram:CategoryCode = $vatCategoryCode`
	`and ram:RateApplicablePercent = $vatRate`
	`]`
	`/ram:BasisAmount`

CII value / attribute	`` `ram:BasisAmount` ; currency: `@currencyID` ``
Selector condition (text)	Select the VAT breakdown entry for the specified VAT category code (`$vatCategoryCode`) and, where present, the VAT rate ($vatRate), ensuring the scheme/type is VAT

`$vatCategoryCode` and `$vatRate` are selector parameters derived from the breakdown record or supplied as instance-level binding inputs. XPath namespace prefixes are declared in the binding specification. In the illustrative **Semantic Path**, `inv` is the module identifier.

D.4 Syntax binding example: XBRL GL tuple palette taxonomy

The current XBRL GL 2.1 tuple design does not provide a header-level VAT breakdown structure analogous to EN 16931 “BG-23 VAT BREAKDOWN” (i.e., per VAT category totals at the document header level). Instead, VAT data is carried at line level in gl-cor:entryDetail/gl-cor:taxes.

- Line-level taxable basis amount

The following XPath selects the taxable basis amount for a given VAT category (and, optionally, rate) at line level:

```
/xbrli:xbrl
  /gl-cor:accountingEntries
  /gl-cor:entryHeader
  /gl-cor:entryDetail
  /gl-cor:taxes[
    gl-cor:taxTableCode          = 'VAT'
    and gl-cor:taxCode           = 'S'
    and gl-cor:taxPercentageRate = 10
  ]
  /gl-cor:taxBasis
```

NOTE 1: Because each XBRL item is identified by its QName, the XPath may be shortened using a descendant selector, for example:

```
//gl-cor:taxes[gl-cor:taxTableCode='VAT' and gl-cor:taxCode='S']/gl-cor:taxBasis
```

- Header-level subtotal (computed)

Because a header-level VAT breakdown is not modelled explicitly, the subtotal for a given VAT category within an entryHeader is obtained by summing the matching line-level taxBasis values:

```
sum(
  /xbrli:xbrl
    /gl-cor:accountingEntries
    /gl-cor:entryHeader
    /gl-cor:entryDetail
    /gl-cor:taxes[
      gl-cor:taxTableCode          = 'VAT'
      and gl-cor:taxCode           = 'S'
      and gl-cor:taxPercentageRate = 10
    ]
    /gl-cor:taxBasis
)
```

NOTE 2: Because each XBRL item is identified by its QName, the XPath may be shortened using a descendant selector, for example:

sum(//gl-cor:taxes[gl-cor:taxTableCode='VAT' and gl-cor:taxCode='S']/gl-cor:taxBasis)

NOTE 3: The descendant selector `//` can match several entries. A binding that addresses one entry scopes the XPath to that entry, for example by `gl-cor:entryNumber`.

Appendix E
References

(Reference list under collaborative review. Official XBRL specifications are available at https://specifications.xbrl.org/.)

RFC2119	The Internet Engineering Task Force (IETF). "Key words for use in RFCs to Indicate Requirement Levels, March 1997" Scott Bradner. (See http://www.ietf.org/rfc/rfc2119.txt)
XPath	World Wide Web Consortium (W3C), XML Path Language (XPath) 2.0 (Second Edition), W3C Recommendation, 14 December 2010. (See https://www.w3.org/TR/xpath20/)
SCHEMA-1	World Wide Web Consortium (W3C). "XML Schema Part 1: Structures" (See http://www.w3.org/TR/xmlschema-1/)
TRP	XBRL International Inc. (XII). "XBRL Taxonomy Recognition Process, 28 October 2004" Peter Calvert, and Joseph MacDonald. (See https://xbrl.org/taxonomyrecognition/taxonomy-recognition-acknowledgement-process-2013-04-09.htm)
XBRL	XBRL International Inc. (XII). "Extensible Business Reporting Language (XBRL) 2.1 Specification with corrected errata dated 2 July 2008" Phillip Engel, Walter Hamscher, Geoff Shuetrim, David vun Kannon, and Hugh Wallis. (See https://specifications.xbrl.org/)
XLINK	World Wide Web Consortium (W3C). "XML Linking Language (XLink) Version 1.0." Steve DeRose, Eve Maler, and David Orchard. (See http://www.w3.org/TR/xlink/)
GXLTFTA	XBRL International Inc. (XII). “XBRL GL Taxonomy Framework Technical Architecture 2015” Gianluca Garbellotto, Eric E. Cohen, Masatomo Goto, Toshimitsu Suzuki (See https://www.xbrl.org/int/gl/2015-03-25/GLTFTA-REC-2015-03-25.html)
XBRL GL Framework	XBRL International Inc. (XII). “XBRL Global Ledger Taxonomy Framework 2015“ Eric E. Cohen, Masatomo Goto, Toshimitsu Suzuki (See https://www.xbrl.org/int/gl/2015-03-25/gl-framework-REC-2015-03-25.html)

Appendix F
Intellectual property status

Copyright and license terms are under review. This draft is made available through the public project repository for collaborative review. Publication of this draft does not establish additional redistribution or derivative-use rights.

References to external specifications and other third-party materials remain subject to the terms of their respective rights holders.

This draft does not claim approval, endorsement, publication, or official status by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

Appendix G
Document history

Current design baseline for collaborative review

Revision 2026-09-15: This section records the current accepted project specification.

Topic	Accepted specification and boundary
Project module namespace	The namespace of module M is exactly https://www.xbrl.or.jp/taxonomy/xbrl-gl-next/M. The URI contains neither an experimental segment nor a date. Namespace prefixes, versioned physical filenames, and directory paths are separate identifiers and do not alter the namespace URI. External XML, XBRL, XLink, ISO, and historical 2015 XBRL GL namespaces remain unchanged.
Model authority and pipeline	The authoritative authored source is the FSM. Specialisation deterministically produces the BSM. Graph Walk produces the candidate LHM. Human review controls the reviewed LHM. Post-Graph Walk produces the root-specific HMDs. The HMD semantic_path, hierarchy, datatype, multiplicity, and value_domain govern Tuple and OIM generation.
Root-specific DTSs	Accounting Entries and Business Transactions are expanded from different root HMDs. Each root is used and validated through its matching Tuple or OIM entry point. The split layout is mandatory; same-named files are not copied over one another and the two DTSs are not mixed without a separate integration design.
Accepted structural differences	Fourteen tracked differences among same-module and same-name declarations are accepted as ROOT_HMD_SPECIFIC_STRUCTURAL_DIFFERENCE. Exact equality of root-derived type, structure, or multiplicity is not required across AE and BT. Each declaration remains subject to its own accepted root HMD and DTS.
Binding responsibility	Part 1 governs the semantic pipeline and formal HMD contract. Part 2 and Part 3 bind each selected HMD independently. UADC consumes semantic_path and HMD constraints through its Binding and Structured CSV layers; Part 1 does not define UADC product mappings.
Current value-domain scope	The accepted HMD contract carries value_domain. EE1 resources are generated from reviewed Value Domain authority. Current Accounting Entries content includes entryDatePosted, taxTransactionClassification (Purchase/Sales), and taxType (VAT/OTH).

Revision 2026-08-29 (reviewed Value Domain authority and implemented EE1): Aligned the lifecycle with the accepted design in which candidate-LHM column 9 remains `association_role`, LHM-reviewed column 9 is `value_domain`, and Post-Graph Walk preserves the reviewed Value Domain unchanged into HMD for taxonomy. Removed the transitional EE1 implementation boundary and documented deterministic EE1 domain/member generation and concept binding.

Revision 2026-08-20 (`gen`, datatype binding, and **Value Domains**): Clarified semantic datatype versus XBRL item type binding; added the shared, binding-neutral `gen` module and **Value Domain** architecture; preserved distinct legacy, standard, and user-defined **Value Domains**; and defined the implementation boundary for companion bindings, crosswalks, external code lists, and Extensible Enumerations.

Revision 2026-08-21 (occurrence identity and EE 1.0 boundary): Distinguished Class hierarchy from occurrence identity; defined occurrence-key Classes from root status and multiplicity; aligned **Structured CSV** occurrence keys with that classification; and stated explicitly that deterministic authoritative **Value Domain**-to-EE 1.0 generation is not yet implemented.

Revision 2026-08-18 (semantic and taxonomy naming clarification): Clarified that LHM `name` is not required to be globally unique; distinguished `name`, `semantic_path`, `local_name`, and QName uniqueness; clarified that taxonomy-name collisions are resolved through `local_name`, not by modifying `name`; and replaced ambiguous wording such as “renamed” and “reviewed names”.

Revision 2026-08-10 v14 (implementation-independent taxonomy generation): Generalised the **conformant taxonomy generator** as a processor of one or more formal **HMDs**; removed manifest, execution-record, hash, diagnostic-count, command, and ancillary-output requirements; simplified **Post-Graph Walk** and taxonomy-generation conformance to observable artefacts and results; and reduced redundant MUST NOT statements while retaining indispensable semantic and structural constraints.

Revision 2026-08-08 v13 (execution records): Removed the separate run-configuration artefact because it had no remaining run-time choices not already determined by the input artefacts and fixed Framework rules. Required execution conditions, implementation versions, outputs, validation results, and file hashes are now recorded in execution records; human **LHM-review** decisions are recorded in an LHM Review Report. Also aligned the formal **HMD_for_taxonomy** treatment of `0..0` occurrence subtrees with the manifest and post_graphwalk workflow.

Revision 2026-08-07 (**local_name** column): Renamed the former LHM syntax-name column to `**local_name**`. Clarified that `**local_name**` is an XML Schema NCName, while an XPath step QName is constructed from the namespace prefix registered for `module` and the row's `local_name`. Revision 2026-08-07 (**HMD** output): Clarified the **Figure 2** explanation so that post_graphwalk has the approved **LHM reviewed** as its sole semantic-data input, partitions that input at each level = 1 root, excludes 0..0 rows from formal output, performs uniqueness checks per **HMD** and definition-consistency checks across **HMDs**, and atomically produces multiple **HMD_for_taxonomy** files with a manifest. Explained that the **Figure 2** box labelled **HMD for taxonomy** denotes this output set and that upstream artefacts are shown only for lifecycle provenance.

Revision 2026-08-07 v12 (occurrence-specific naming): Clarified that assigning binding-specific `local_name` values according to occurrence position is a responsibility of **LHM review**; that equality of `source_bsm_id` does not require equality of reviewed QNames, type names, or child QNames; and that a `local_name` adjustment does not change `name` or `semantic_path`. Aligned the duplicate and reuse checks with root-specific **HMD** processing, retained deliberate shared declarations across **HMDs** only when their authoritative definitions are identical, distinguished palette-specific `cor` and `btx` declarations from shared `bus`, `muc`, `emh`, and `tax` modules, and clarified same-namespace include and cross-namespace import behaviour.

Revision 2026-08-06: Aligned **Figure 2** and all **LHM** clauses with the 18-column **Graph Walk** output, `local_name`-only review adjustments, business-requirement-based multiplicity restrictions, and **post_graphwalk** XPath regeneration. Distinguished the three semantic-model layers from their lifecycle artefacts; removed the obsolete `authoritative LHM` lifecycle term; added definitions for **FSM source**, **BSM result**, **candidate LHM**, review responsibility, **post-Graph Walk** checks, and taxonomy completion; and aligned **HMD**, conformance, and annex terminology accordingly.

Revision 2026-08-03: Aligned terminology, **FSM/BSM/LHM** contracts, **Specialisation, Graph Walk**, Aligned Pool, **post_graphwalk**, and conformance clauses with the revised Requirements Specification; removed QName-candidate checks from **LHM** authority; and replaced the binding-specific **LHM** `domain_name` column with the serialisation-neutral `association_role` column.

Revision 2026-08-02: Distinguished **Specialisation** from **Graph Walk** throughout the normative text and examples; made the principal **Specialisation** and **Graph Walk** processing rules normative; corrected Reference Association and Appendix B behaviour; defined the post_graphwalk taxonomy-definition boundary; clarified **BSM** and **LHM** columns; and defined the Framework-specific **Semantic Path** construction rules.

Revision 2026-08-01 (**FSM/BSM/LHM** alignment): The **FSM** has 15 columns, the **BSM** adds `id` as its sixteenth column, and the **LHM** has 16 columns. The **LHM** uses `semantic_path` as its row identifier and `source_bsm_id` for traceability to the originating **BSM** row.

Revision 2026-08-01 (syntax-name constraints): Each **candidate LHM** `semantic_path` corresponds to one `source_bsm_id`. One `source_bsm_id` may occur on multiple rows having different unique `semantic_path` values. Syntax-name candidates are checked after **Graph Walk** and before taxonomy generation.

Revision 2026-07-28: Established **FSM** authority, **Specialisation**, **Graph Walk**, module and Palette concepts, and extension and build records. Later revisions make the conformance rules implementation-independent.

The following statements define the current technical-validation baseline for this collaborative review.

• Semantic model: The semantic model consists of the 15-column **FSM**, 16-column **BSM**, and 18-column **LHM**. The **BSM** uses `id`; the **LHM** uses `semantic_path` as its row identifier and `source_bsm_id` for **BSM** traceability. The LHM also contains mechanically generated `local_name` and `xpath` taxonomy-preparation values that do not define semantic identity.

• Semantic-model identifiers and row traceability: A Class, Attribute, or Association is identified by its semantic-model identifier. The **BSM** assigns a unique `id` to each effective row. **Graph Walk** copies that value to LHM `source_bsm_id`; `semantic_path` identifies the LHM row. Each `semantic_path` corresponds to exactly one `source_bsm_id`; one `source_bsm_id` may be reused on multiple rows having different unique `semantic_path` values.

• LHM and syntax binding: Graph Walk produces a **candidate LHM** with `association_role` in column 9 and initial `local_name` and `xpath`. After semantic-model approval, the controlled LHM review replaces column 9 with `value_domain`, assigns applicable reviewed Value Domain identifiers, records permitted `local_name` adjustments and business-requirement-based `multiplicity` restrictions, and preserves all other approved semantic content. post_graphwalk preserves `value_domain` unchanged, regenerates every `xpath`, performs the post-Graph Walk checks, and produces the HMD for taxonomy.

• Shared, Aligned, and Distinct: Shared definitions support broad cross-domain reuse. Aligned definitions preserve alignment with a stated public context, law, regulation, standard, process, or document structure. Distinct definitions serve an enterprise, group, product, bilateral relationship, or closed community. Classification depends on meaning and reuse scope.

• Property and Association processing: Duplicate and conflict checks use the applicable semantic-model identifier. Association type and multiplicity are content fields. Multiplicity `0` or `0..0` applies a deletion directive during **Specialisation**.

• Syntax-name processing: **Graph Walk** generates the provisional `local_name` from `name`; `module` supplies the namespace context used to construct the QName. The responsible model definer assigns occurrence-appropriate binding-specific `local_name` values only in the **LHM reviewed**. The same `source_bsm_id` may therefore have different reviewed `local_name` values and QNames at different occurrence positions. A `local_name` adjustment does not change `name` or `semantic_path`. post_graphwalk preserves the reviewed `local_name`, checks QName uniqueness within each HMD and definition consistency for shared QNames across HMDs, and regenerates `xpath` separately within each HMD.

• Artefact status: The taxonomy date `2026-12-31` is provisional and indicates the 2026 technical-validation baseline. Formal publication status depends on the applicable approval process.

Date	Details
6 January 2026	v0.1 Initial collaborative review draft.

Date	Details
28 July 2026	Clarified **FSM** authority; detailed **Specialisation** and **Graph Walk**; defined **HMD** roots, message entry points, module composition, and the Palette model; introduced extension and build-record responsibilities.
29 July 2026	Aligned the **FSM** and **BSM** information structures and Association comparison rules with **FSM**_q.xlsx; distinguished association_role from property_term; retained associated_module and associated_class; removed the obsolete **BSM** field; clarified the information required in the LHM; and specified non-fatal diagnostics and continued processing when the target of a Reference Association has no primary key.
1 August 2026	Clarified the roles, required information, and processing boundaries of the **FSM**, **BSM**, and **LHM**; defined the **BSM** id as the source identifier retained in the LHM as source_bsm_id; limited **Graph Walk** to producing a **candidate LHM** from the domain-wide **BSM**; defined duplicate and conflict checks for the final **BSM** and LHM; defined Classes, Attributes, and Associations using their semantic-model identifiers; and separated read-only syntax-name candidate reporting and subsequent XBRL syntax binding from **Graph Walk**.
2 August 2026	Defined syntax-independent semantic-model identifiers and their **FSM**/**BSM** column representations; distinguished the **Specialisation** process from the **Specialisation** Association type; made conformance implementation-independent and artefact-based; aligned post-LHM taxonomy generation; and expressed conformance conditions positively.
5–6 August 2026	Defined LHM artefacts, revised **Figure 2** and post_graphwalk flow, and v0 domain_name exclusion.
8 August 2026	Removed the separate run-configuration artefact from Part 1 because processing choices are fixed by the Framework and applicable binding Parts; introduced execution records and the LHM Review Report; aligned roots, taxonomy assembly, Palette, syntax binding, conformance, and annex terminology; and resolved the 0..0-row inconsistency by excluding each such occurrence subtree from formal **HMD_for_taxonomy** output while recording it in the manifest.
10 August 2026	Generalised **Post-Graph Walk** and taxonomy generation; removed required manifests, execution records, hashes, diagnostic counts, and ancillary output; and simplified prohibitions to indispensable conformance constraints.
14 August 2026	Defined canonical **Semantic Path** term normalisation as retention of ASCII letters only; clarified direct Association-role and associated-Class concatenation and use of LHM module identifiers; prohibited empty normalised terms and normalisation collisions; required **Post-Graph Walk** processing to validate and preserve semantic_path without repair or regeneration; and corrected the candidate-LHM review table so semantic_path is not manually adjustable.
28 August 2026	Phase 2C: defined **HMD for taxonomy** column 9 as `value_domain`; adopted deterministic EE1 generation; clarified selector-effective multiplicity without rewriting accepted HMD multiplicity; established one-HMD/one-DTS validation and the minimum-tree program/Generator authority.
29 August 2026	Aligned the LHM-review contract with the accepted Value Domain design: **candidate LHM** column 9 remains `association_role`, LHM reviewed uses `value_domain`, and Post-Graph Walk preserves it unchanged into HMD for taxonomy. Updated deterministic EE1 generation wording and removed obsolete transitional implementation statements.
15 September 2026	Restored the accepted 2026-08-29 successor as the base; aligned project module namespaces with the stable URI; recorded root-HMD-specific AE/BT split conformance and the accepted 14 structural differences; aligned the UADC interface boundary and current semantic items where applicable.

Appendix H
Errata corrections in this document

This appendix records editorial corrections incorporated into this collaborative review draft. The entries have not been approved or endorsed by XBRL Japan, XBRL Europe, XBRL International, or any other organization.

No errata have been incorporated into this document.

Annex — Correspondence with XBRL GL Next Requirements Specification (informative)

The requirement identifiers below are taken exclusively from the supplied XBRL GL Next Requirements Specification. This correspondence identifies relevant requirements, not a separate conformance assertion. Technical clauses in this Part remain binding-specific or implementation-detail requirements as written; no new GL-REQ identifier is established by this annex.

Clause	Relevant requirements	Correspondence basis
2 Introduction	GL-REQ-0001, GL-REQ-0201, GL-REQ-0301	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
2.1 Scope of this document	GL-REQ-0001, GL-REQ-0201, GL-REQ-0301	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
2.2 Normative language and document conventions	GL-REQ-0001, GL-REQ-0201, GL-REQ-0301	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
2.3 Relationship to other Parts	GL-REQ-0001, GL-REQ-0201, GL-REQ-0301	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3 Terms and definitions	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3.1 General concepts	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3.2 Semantic-model constructs	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3.3 Semantic-model layers and lifecycle artefacts	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3.4 Semantic processing	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
3.5 Review, conformance, and taxonomy completion	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are

		retained (not an independent requirement ID).
3.6 Modules and taxonomy assembly	GL-REQ-0001, GL-REQ-0102, GL-REQ-0201	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
4 Requirements and Motivation	GL-REQ-0002, GL-REQ-0004, GL-REQ-0201, GL-REQ-0301	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5 Framework Architecture Overview	GL-REQ-0001, GL-REQ-0009, GL-REQ-0201, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.1 Semantic-model layers and processing pipeline	GL-REQ-0001, GL-REQ-0009, GL-REQ-0201, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.2 Processing pipeline	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.2.1 Specialisation (FSM → BSM)	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.2.2 Graph Walk (BSM → LHM)	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3 FSM, BSM, and LHM definition tables	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.1 General	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.2 Foundational Semantic Model (FSM)	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.3 Business Semantic Model (BSM)	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

5.3.4 Multiple inheritance in the BSM	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.5 Logical Hierarchical Model (LHM)	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.6 Relationship among the three tables	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.7 LHM review after Graph Walk	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.8 Post-Graph Walk processing	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.9 HMD for taxonomy definition table	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.10 Relationship among candidate LHM, LHM reviewed, and HMD for taxonomy	GL-REQ-0001, GL-REQ-0009, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.11 Datatype binding and Value Domain generation	GL-REQ-0301, GL-REQ-0501, GL-REQ-0502	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.3.12 Class hierarchy and occurrence identity	GL-REQ-0304	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.4 Modules and taxonomy assemblies	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.4.1 Module types	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.4.2 Use-case taxonomy assemblies	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

5.4.3 Semantic and syntax bindings	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.4.4 `gen` module and Value Domains	GL-REQ-0501, GL-REQ-0502	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.5 XBRL QName and concept naming	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.5.1 XBRL concepts and QName	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.5.2 Prohibited and corrected example	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.5.3 `local_name` reuse and duplicate checks	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.6 Semantic Path	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.6.1 Purpose and Reference	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.6.2 Syntax Rules and Construction	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.6.3 Functional Role in the Framework	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
5.6.4 Conformance and Validation	GL-REQ-0009, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
6 Extension Mechanisms and Governance	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

6.1 Semantic authority and extension point	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
6.2 Shared, Aligned, and Distinct	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
6.3 Namespace, ownership, and versioning	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
6.4 Property matching, duplicates, and conflicts	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
6.5 Aligned Pool governance	GL-REQ-0011, GL-REQ-0202, GL-REQ-0204, GL-REQ-0208, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7 Specialisation and Graph Walk	GL-REQ-0001, GL-REQ-0209, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.1 Scope, terminology, and processing stages	GL-REQ-0001, GL-REQ-0209, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2 Specialisation: FSM to BSM	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.1 Inputs and registration	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.2 Semantic identifier and Class-reference resolution	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.3 Inheritance resolution	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.3.1 Cycle validation	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

7.2.3.2 Resolution flow	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.4 Property matching and duplicate-check columns	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.5 Override, addition, and deletion	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.6 Abstract Classes and invalid Associations	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.7 BSM output contract	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.2.8 Specialisation conformance	GL-REQ-0001, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3 Graph Walk: BSM to LHM	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.1 Inputs and root selection	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.2 Class map and property order	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.3 Depth-first traversal	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.3.1 Traversal state	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.3.2 Traversal flow	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

7.3.4 Association semantics	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.5 Cycles and repeated occurrences	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.6 Semantic path	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.7 Boundary between Graph Walk and post_graphwalk processing	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.8 LHM output contract	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.3.9 Graph Walk conformance	GL-REQ-0001, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.4 Later root-specific syntax binding (HMD)	GL-REQ-0001, GL-REQ-0209, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.5 Implementation independence	GL-REQ-0001, GL-REQ-0209, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
7.6 Conformance test coverage (informative)	GL-REQ-0001, GL-REQ-0209, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
8 Modules and the Palette Model	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
8.1 Module concept	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
8.2 Baseline module composition	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

8.3 Module dependencies and ownership	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
8.4 Taxonomy assembly realisation	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
8.5 Palette model	GL-REQ-0201, GL-REQ-0207, GL-REQ-0210	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
9 Palette Taxonomy	GL-REQ-0301, GL-REQ-0302	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
9.1 Purpose	GL-REQ-0301, GL-REQ-0302	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
9.2 Tuple palette taxonomy	GL-REQ-0301, GL-REQ-0302	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
9.3 OIM-oriented taxonomy	GL-REQ-0301, GL-REQ-0302	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
9.4 Message and DTS entry points	GL-REQ-0301, GL-REQ-0302	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
10 How to create extensions within the XBRL GL Next Taxonomy Framework	GL-REQ-0011, GL-REQ-0203, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
10.1 Define the semantic extension	GL-REQ-0011, GL-REQ-0203, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
10.2 Validate and specialise	GL-REQ-0011, GL-REQ-0203, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
10.3 Generate hierarchies	GL-REQ-0011, GL-REQ-0203, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

10.4 Generate taxonomy bindings	GL-REQ-0011, GL-REQ-0203, GL-REQ-0209	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
11 Syntax Binding	GL-REQ-0301, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
11.1 Binding source	GL-REQ-0301, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
11.2 Binding information	GL-REQ-0301, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
11.3 XBRL 2.1 and OIM equivalence	GL-REQ-0301, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
11.4 Validation	GL-REQ-0301, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12 Reverse engineering and migration support (informative)	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.1 Purpose	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.2 Background: XBRL GL XML (XBRL 2.1 tuple-based realization) and implicit semantics	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.3 Principle of semantic primacy	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.4 Role of the Foundational Semantic Model (FSM)	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.5 Reconstruction of semantic layers	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).

12.6 Regeneration and dual syntax support	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
12.7 Summary	GL-REQ-0205, GL-REQ-0303	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13 Conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.1 General	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.2 FSM conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.3 BSM conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.4 LHM conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.5 Execution-record and post_graphwalk conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).
13.6 Processing conformance	GL-REQ-0107, GL-REQ-0401, GL-REQ-0403	Related scope; clause-specific technical provisions are retained (not an independent requirement ID).